6CCSANLP

Natural Language Processing

Revision Notes

Academic Year 2025/26 — Semester 1

Compiled on January 2, 2026

These notes are structured to help you prepare for exams.
Each week includes: learning objectives, core concepts, theory, and worked examples.

Contents

Module Overview **6**

1 Week 1: Introduction to NLP and Text Preprocessing **7**
1.1 Topic Summary . . . 7
1.2 Overview + Core Concepts + Extended Theory . . . 7
1.2.1 Tokens vs Types . . . 8
1.2.2 What Counts as a Word? . . . 8
1.2.3 Morphemes: The Minimal Units of Meaning . . . 9
1.2.4 Morphological Typology (Brief Overview) . . . 9
1.2.5 Unicode and UTF-8 . . . 10
1.2.6 Heaps'/Herdan's Law . . . 11
1.2.7 Byte-Pair Encoding (BPE) . . . 11
1.2.8 BPE Across Languages . . . 12

2 Week 2: Text Representation Techniques **13**
2.1 Topic Summary . . . 13
2.2 Overview + Core Concepts + Extended Theory . . . 13
2.2.1 Lexical Semantics: Desiderata for Word Meaning . . . 14
2.2.2 The Distributional Hypothesis . . . 14
2.2.3 Vector Semantics: Words as Points in Space . . . 14
2.2.4 Co-occurrence Matrices . . . 15
2.2.5 Similarity Metrics: Dot Product and Cosine . . . 16
2.2.6 Weighting: tf-idf . . . 16
2.2.7 Dense Embeddings: Word2Vec . . . 17
2.2.8 Properties of Embeddings . . . 19
2.2.9 Summary: Sparse vs. Dense Methods . . . 19

3 Week 3: Statistical Language Models **20**
3.1 Topic Summary . . . 20
3.2 Overview + Core Concepts + Extended Theory . . . 20
3.2.1 What is a Language Model? . . . 21
3.2.2 The Chain Rule of Probability . . . 21
3.2.3 The Markov Assumption . . . 22
3.2.4 N-gram Models . . . 22
3.2.5 Maximum Likelihood Estimation (MLE) . . . 22
3.2.6 Practical Issues: Log Probabilities . . . 23
3.2.7 Sentence Boundary Tokens . . . 24
3.2.8 Evaluation: Train / Dev / Test Sets . . . 24
3.2.9 Perplexity . . . 24
3.2.10 Sampling from N-gram Models . . . 25
3.2.11 Sparsity and Zeros . . . 25

3.2.12 Smoothing: Add-One (Laplace) . . . 26
3.2.13 Interpolation and Backoff . . . 27
3.2.14 Limitations of N-gram Models . . . 27
3.2.15 Summary: N-gram Language Models . . . 28

4 Week 4: Neural Language Models, RNNs, and LSTMs 29
4.1 Topic Summary . . . 29
4.2 Overview + Core Concepts + Extended Theory . . . 30
4.2.1 Embeddings and the Embedding Matrix . . . 30
4.2.2 Embeddings as Input to Feedforward Classifiers . . . 31
4.2.3 Feedforward Neural Language Models . . . 31
4.2.4 Neural LMs vs. N-gram LMs . . . 32
4.2.5 Recurrent Neural Networks (RNNs) . . . 32
4.2.6 RNNs as Language Models . . . 33
4.2.7 Training RNNs . . . 34
4.2.8 RNNs for NLP Tasks . . . 35
4.2.9 Long Short-Term Memory Networks (LSTMs) . . . 36
4.2.10 Encoder-Decoder Architecture . . . 37
4.2.11 Attention Mechanism . . . 38
4.2.12 Summary: Neural Architectures for Sequences . . . 39

5 Week 5: Transformer Models and Large Language Models 40
5.1 Topic Summary . . . 40
5.2 Overview + Core Concepts + Extended Theory . . . 41
5.2.1 Introduction to Large Language Models . . . 41
5.2.2 Three LLM Architectures . . . 42
5.2.3 Pretraining Large Language Models . . . 42
5.2.4 Training Data and Its Problems . . . 43
5.2.5 Evaluating LLMs: Perplexity . . . 43
5.2.6 Static vs Contextual Embeddings . . . 44
5.2.7 Self-Attention Mechanism . . . 44
5.2.8 Multi-Head Attention . . . 45
5.2.9 The Transformer Block . . . 46
5.2.10 Parallelising Attention and Causal Masking . . . 47
5.2.11 Positional Encodings . . . 47
5.2.12 Language Modelling Head . . . 48
5.2.13 Scaling Laws and KV Cache . . . 48
5.2.14 Summary: Transformers vs RNNs . . . 49

6 Week 7: Prompting and In-Context Learning **50**
6.1 Topic Summary . . . 50
6.2 Overview + Core Concepts + Extended Theory . . . 50
6.2.1 LLM Recap: Auto-Regressive Generation . . . 51
6.2.2 Interface vs. API . . . 51
6.2.3 Introduction to Prompt Engineering . . . 52
6.2.4 Vanilla Prompting Strategies . . . 52
6.2.5 In-Context Learning (ICL) . . . 53
6.2.6 Chain-of-Thought (CoT) Prompting . . . 53
6.2.7 Self-Reflection . . . 54
6.2.8 Temperature and Self-Consistency . . . 54
6.2.9 Tool Use . . . 55
6.2.10 Multi-Agent Systems . . . 56
6.2.11 Summary: Prompting Techniques . . . 56

7 Week 8: LLM Training Strategies **57**
7.1 Topic Summary . . . 57
7.2 Overview + Core Concepts + Extended Theory . . . 57
7.2.1 Challenges in LLM Training . . . 58
7.2.2 The Three-Step LLM Training Pipeline . . . 58
7.2.3 Pre-training: Next-Token Prediction . . . 59
7.2.4 Memorisation vs. Learning . . . 59
7.2.5 Instruction Fine-Tuning (SFT) . . . 60
7.2.6 SFT Challenges . . . 60
7.2.7 Self-Instruct: Generating Training Data . . . 61
7.2.8 Reinforcement Learning from Human Feedback (RLHF) . . . 61
7.2.9 Reward Models . . . 62
7.2.10 PPO: Proximal Policy Optimization . . . 62
7.2.11 LoRA: Low-Rank Adaptation . . . 63
7.2.12 DPO: Direct Preference Optimization . . . 64
7.2.13 GRPO: Group Relative Policy Optimization . . . 64
7.2.14 Summary: Training Methods Comparison . . . 64
7.2.15 Key Equations Summary . . . 65

8 Week 9: Retrieval-Augmented Generation (RAG) **66**
8.1 Topic Summary . . . 66
8.2 Overview + Core Concepts + Extended Theory . . . 66
8.2.1 What is RAG? . . . 67
8.2.2 RAG vs. Fine-tuning . . . 67
8.2.3 RAG Application Scenarios . . . 67
8.2.4 Information Retrieval Foundations . . . 68
8.2.5 Discrete Text Representations . . . 68

8.2.6 Continuous Text Representations (Dense Embeddings) . . . 68
8.2.7 Similarity Metrics . . . 69
8.2.8 Indexing for Efficient Retrieval . . . 70
8.2.9 IR Evaluation Metrics . . . 70
8.2.10 Naïve RAG Pipeline . . . 71
8.2.11 Advanced RAG Enhancements . . . 71
8.2.12 When and How to Retrieve . . . 72
8.2.13 Key Technologies for RAG Optimisation . . . 72
8.2.14 Summary: RAG Pipeline Comparison . . . 73
8.2.15 Key Formulas Summary . . . 73

9 Week 10: Games and LLM Reasoning **74**
9.1 Topic Summary . . . 74
9.2 Overview + Core Concepts + Extended Theory . . . 74
9.2.1 Definition of Reasoning . . . 75
9.2.2 Game-Based Reasoning: From Pacman to Go . . . 75
9.2.3 Complete vs. Incomplete Information Games . . . 75
9.2.4 AlphaGo: Learning to Play Go . . . 76
9.2.5 Connecting Games to LLM Reasoning . . . 77
9.2.6 Why Intermediate Tokens (Chain-of-Thought) Matter . . . 77
9.2.7 Supervised Fine-Tuning (SFT) for Reasoning . . . 78
9.2.8 Reinforced Fine-Tuning (RLFT) . . . 78
9.2.9 Self-Consistency Decoding . . . 79
9.2.10 Latent Reasoning (Advanced) . . . 79
9.2.11 Reasoning with Retrieval . . . 79
9.2.12 Summary: Reasoning Methods Comparison . . . 80

10 Week 11: LLM Evaluation and Safety **81**
10.1 Topic Summary . . . 81
10.2 Overview + Core Concepts + Extended Theory . . . 81
10.2.1 Why Evaluation Matters . . . 82
10.2.2 Evaluation Tasks . . . 82
10.2.3 Evaluation Benchmarks . . . 82
10.2.4 Evaluation Methods . . . 83
10.2.5 Evaluation Metrics . . . 83
10.2.6 LLM-as-a-Judge . . . 84
10.2.7 Evaluation Challenges . . . 84
10.2.8 LLM Safety Overview . . . 85
10.2.9 Social Bias in LLMs . . . 85
10.2.10 Toxicity in LLMs . . . 86
10.2.11 Jailbreaking Attacks . . . 86
10.2.12 Defense Strategies . . . 87
10.2.13 Summary Tables . . . 87

Module Overview

About This Module

This module introduces students to **Natural Language Processing (NLP)** and the computational methods for processing human language, as well as the experimental methodology necessary for applying NLP in statistically valid settings.

Module code: 6CCSANLP **Semester:** 1 (AY 2025/26)

Module Aims

The module aims to:

- Provide an introduction to NLP and cover computational methods for manipulating human language.
- Cover core tasks in NLP such as **sentiment analysis** and **machine translation**.
- Teach students how to **determine appropriate methods** for a range of NLP tasks.
- Develop skills to **implement** and **evaluate** NLP models.

Learning Outcomes

By the end of this module, you will be able to:

Code	Learning Outcome
NLP1	Understand the broad range of **methodologies and algorithms** related to language processing.
NLP2	**Analyse and formulate** the objectives of different NLP problems.
NLP3	Apply algorithms in **statistically valid experiments**.
NLP4	Determine appropriate methods and know how to **implement them**.
NLP5	Implement **machine learning algorithms** for NLP using a high-level programming language.
NLP6	**Assess the success (or failure)** of an algorithm to learn a particular task.

---

1 Week 1: Introduction to NLP and Text Preprocessing

1.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Differentiate **word tokens** and **word types**, and explain why the distinction matters.
2. Recognise tokenisation challenges: contractions, disfluencies, punctuation, cross-linguistic variation.
3. Explain **Heaps'/Herdan's Law** and why vocabulary size grows with corpus size.
4. Describe **Unicode** and **UTF-8** as standards for representing text.
5. Understand **subword tokenisation (BPE)** as a solution to open-vocabulary problems.

The Big Picture

This week addresses a deceptively simple question: *"What is a unit of text?"*

The answer is surprisingly complex:

- "Words" are not universally defined (some languages don't use spaces).
- Vocabulary grows unboundedly with more text data.
- Modern NLP uses **subword** units instead of whole words.

What Examiners Like to Ask

- Explain or compute **types vs tokens** on an example sentence; discuss case/punctuation choices.
- Identify tokenisation challenges: contractions, disfluencies, no-whitespace languages.
- State and apply **Heaps' Law**; interpret the exponent b.
- Unicode/UTF-8 basics: code points vs bytes, ASCII compatibility, typical byte lengths.
- BPE algorithm: what is merged, **train vs test time** behaviour, why it helps with unknown words.

1.2 Overview + Core Concepts + Extended Theory

Roadmap

1. What a "word" can mean (tokens, types, morphemes)
2. How text is stored (Unicode and UTF-8)
3. Why unknown words are inevitable (vocabulary explosion)
4. Heaps'/Herdan's Law (formalises vocabulary growth)
5. Byte-Pair Encoding (BPE): training and inference

1.2.1 Tokens vs Types

Tokens vs Types

Token (instance): A single occurrence of a word/unit in running text.

Type: A distinct vocabulary item. The vocabulary size is denoted $|V|$.

Why it matters:

- Probabilities and counts behave differently for tokens (occurrences) vs types (unique forms).
- "How many words in a sentence?" depends on whether you mean tokens or types.

Example: Counting Tokens and Types

Sentence: "They picnicked by the pool, then lay back on the grass and looked at the stars."

Analysis (ignoring punctuation):

- **Tokens:** 16 (count each word occurrence)
- **Types:** 14 ("the" appears 3 times but counts as 1 type)

Edge cases to consider:

- Are "They" and "they" the same type? (Case-folding decision)
- Do we count punctuation marks as tokens?

Exam Tip: Token/Type Questions

When asked "how many words", immediately clarify:

- Punctuation as tokens?
- Case-folding (They = they)?
- Contractions split (I'm → I + 'm)?
- Speech fillers kept or removed?

1.2.2 What Counts as a Word?

Defining "word" is harder than it seems:

Challenge	Example
Contractions	`I'm` = 1 orthographic token, but 2 grammatical words (`I` + `am`)
Disfluencies	Speech contains fragments (`main-`) and fillers (`uh`, `um`)
Punctuation	Is "," a word? Is "..." a word?
No whitespace	Chinese, Japanese, Thai have no spaces between words

Example: Contractions

Question: "`I'm`" is orthographically one token in English. Grammatically it corresponds to how many words?

Answer: Two words:

1. `I` — subject pronoun
2. `'m` (short for `am`) — auxiliary verb

Cross-Linguistic Awareness

Chinese tokenisation is especially tricky:

- Each character (hanzi) represents a morpheme.
- Words average 2.4 characters, but word boundaries are disputed.
- Different standards (Chinese Treebank, Peking University) segment differently.
- Some systems just use characters as tokens.

1.2.3 Morphemes: The Minimal Units of Meaning

Morpheme

A **morpheme** is a minimal meaning-bearing unit in a language.

- `fox` = 1 morpheme
- `cats` = 2 morphemes: `cat` + `-s` (plural marker)

Types of morphemes:

Type	Role	Example
Root	Main meaning	`work` in `worked`
Inflectional	Grammatical features	`-ed` (past), `-s` (plural)
Derivational	Changes word class	`care` (N) → `careful` (Adj) → `carefully` (Adv)

Clitic

A **clitic** is a morpheme that:

- Behaves syntactically like a word, but
- Cannot stand alone (reduced form, attached to another word)

Examples:

- English: `'ve` in `I've` (can't say *"ve")
- English: `'s` in `the teacher's book`
- French: `l'` in `l'opéra`

1.2.4 Morphological Typology (Brief Overview)

Languages vary along two dimensions relevant to tokenisation:

Dimension	Low End	High End
Morphemes per word	**Analytic** (few): Chinese, Vietnamese	**Polysynthetic** (many): Inuit
Segmentability	**Agglutinative** (clean boundaries): Turkish	**Fusional** (features packed): Russian, English

Exam Tip: Typology Pairing

Agglutinative = clean morpheme boundaries (easy to segment).
Fusional = multiple grammatical features packed into one affix (hard to segment).

1.2.5 Unicode and UTF-8

Unicode

Unicode is a standard that assigns a unique **code point** to every character across all scripts.

- Over 150,000 characters from 168+ scripts
- Code points written as `U+XXXX` (hexadecimal)
- First 127 code points = ASCII (backwards compatible)

UTF-8 Encoding

UTF-8 is a variable-length *encoding* of Unicode code points into bytes.
Typical byte lengths:

Script/Character Type	Bytes in UTF-8
ASCII (English letters, digits)	1
Most European scripts (accents, etc.)	2
Most CJK characters	3
Rare CJK / Emoji	4

Example: ASCII and UTF-8

Question: Which string is ASCII-only (identical in UTF-8)?

- `"hello"` — **Yes**, ASCII-only → same bytes in UTF-8
- `"Señor"` — No, contains accented characters (non-ASCII)
- Chinese characters — No (would require multi-byte UTF-8)

Python 3 Note

In Python 3:

- Strings are sequences of Unicode *code points*.
- `len("hello")` returns 5 (code points, not bytes).
- Files must be encoded/decoded when reading/writing.

1.2.6 Heaps'/Herdan's Law

Heaps'/Herdan's Law (Empirical)

$$|V| = kN^b$$

where:

- $|V|$ = vocabulary size (number of types)
- N = corpus size (number of tokens)
- $k > 0$ and $0 < b < 1$ are constants (typically $b \approx 0.5$)

Interpretation: Vocabulary grows *sublinearly* with corpus size. As you collect more text, you keep discovering new word types—but at a decreasing rate.

Example: Scaling with Heaps' Law

Given: $|V| = kN^{0.5}$

Question: If we quadruple the corpus size ($N \to 4N$), how does vocabulary change?

Solution:

$$\frac{|V_{\text{new}}|}{|V_{\text{old}}|} = \frac{k(4N)^{0.5}}{kN^{0.5}} = \sqrt{4} = 2$$

Answer: Vocabulary size **doubles**.

Why This Matters

No matter how big your vocabulary, there will always be **unknown words**. This motivates **subword tokenisation**.

1.2.7 Byte-Pair Encoding (BPE)

BPE is the dominant subword tokenisation algorithm in modern NLP (used by GPT, BERT variants, etc.).

BPE Token Learner (Training)

Input: Training corpus, number of merges k

Algorithm:

1. Initialise vocabulary V with all unique characters/bytes.
2. Repeat k times:
   (a) Find the most frequent adjacent pair (t_L, t_R) in the corpus.
   (b) Create new token $t_{\text{new}} = t_L t_R$ and add to V.
   (c) Replace all occurrences of (t_L, t_R) with t_{new}.

Note: Merges typically happen *within* word boundaries (no cross-word merges).

Example: BPE Training on a Toy Corpus

Corpus: `set new new renew reset renew`

After preprocessing (with space markers):

Word	Count
_new	2
_renew	2
_set	1
_reset	1

Initial vocabulary: _, e, n, r, s, t, w.

First merge: n e → ne (count = 4: appears in both new and renew)

Subsequent merges:

- ne w → new
- _ r → _r
- _r e → _re (learns the prefix re-!)

BPE Encoder (Test Time)

Input: New text to tokenise

Algorithm:

1. Split the input into characters/bytes.
2. Apply the learned merge rules **greedily, in the order they were learned**.
3. Return the resulting token sequence.

Critical point: Test-set frequencies are **not used**. Merges follow the fixed learned order.

Why BPE Works

- **No unknown tokens:** At worst, falls back to individual bytes (only 256 values).
- **Learns meaningful subwords:** Frequent morphemes (like -ing, re-) become tokens.
- **Deterministic:** Same input always produces same tokenisation.

1.2.8 BPE Across Languages

Tokenisation Inequality

Even multilingual BPE tokenisers are trained mostly on English data.

Result: English words are often single tokens, while other languages get split into many pieces.

Example (GPT-4o tokenisation):

- English recipe sentence: 18 tokens (14 words, no splits)
- Spanish translation: 33 tokens (16 words, many splits like h ondo for "deep")

This affects inference cost and potentially model quality for non-English languages.

---

2 Week 2: Text Representation Techniques

2.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Explain the **distributional hypothesis**: words in similar contexts have similar meanings.
2. Represent words/documents as vectors using **term-document** and **word-word** co-occurrence matrices.
3. Compare word vectors using **dot product** and **cosine similarity**.
4. Describe weighting methods (**tf-idf**, **PMI**) and why they improve on raw counts.
5. Understand the motivation for **dense embeddings** (e.g., Word2Vec) over sparse count vectors.

The Big Picture

This week answers: *"How can we represent word meaning computationally?"*

Key insight: instead of treating words as opaque symbols, we represent them as **vectors** in a high-dimensional space where **similar meanings** correspond to **nearby points**.

- **Count-based**: build vectors from co-occurrence statistics (sparse, interpretable).
- **Prediction-based**: learn vectors by training a classifier (dense, generalises better).

What Examiners Like to Ask

- State the **distributional hypothesis** and give an example.
- Explain why **raw dot product** is problematic; derive or apply **cosine similarity**.
- Compute or interpret **tf-idf** weights (especially why common words get weight ≈ 0).
- Describe **Skip-Gram with negative sampling**: training objective, role of sigmoid, why scalable.
- Discuss **bias in embeddings** and the **parallelogram analogy** method (and its caveats).

2.2 Overview + Core Concepts + Extended Theory

Roadmap

1. Desiderata for word meaning (synonymy, similarity, relatedness, connotation)
2. The distributional hypothesis
3. Co-occurrence matrices: term-document vs. word-word
4. Similarity metrics: dot product and cosine
5. Weighting schemes: tf-idf
6. Dense embeddings: Word2Vec (Skip-Gram with negative sampling)
7. Properties, analogies, and bias in embeddings

2.2.1 Lexical Semantics: Desiderata for Word Meaning

Key Relations Between Word Senses

Relation	Description / Example
Polysemy	One lemma, multiple senses: `mouse` (rodent vs. device)
Synonymy	Same meaning in some contexts: `couch`/`sofa`, `car`/`automobile`
Antonymy	Opposites differing on one feature: `hot`/`cold`, `up`/`down`
Similarity	Share meaning elements but not synonyms: `car`/`bicycle`
Relatedness	Associated via semantic field: `coffee`/`cup` (related, not similar)
Connotation	Affective meaning: `copy` (neutral) vs. `knockoff` (negative)

Connotation dimensions (Osgood et al., 1957):

- **Valence**: pleasantness (`love` = high, `toxic` = low)
- **Arousal**: emotional intensity (`frenzy` = high, `mellow` = low)
- **Dominance**: control exerted (`powerful` = high, `weak` = low)

Exam Tip: No Perfect Synonyms

Even near-synonyms differ in register, formality, or connotation (e.g., `water` vs. H_2O).

2.2.2 The Distributional Hypothesis

Distributional Hypothesis

"Words that occur in similar contexts tend to have similar meanings."

— Zellig Harris (1954); Firth (1957)

Implication: We can infer word meaning from *usage patterns* rather than dictionary definitions.

Example: Inferring Meaning from Context

Suppose you encounter the unknown word `ongchoi` in:

- "Ongchoi is delicious sautéed with garlic."
- "Ongchoi leaves with salty sauces."

Compare with known contexts for `spinach`, `chard`, `collard greens`.

Conclusion: `ongchoi` is a leafy green vegetable — inferred purely from shared context words.

2.2.3 Vector Semantics: Words as Points in Space

Core Idea

Represent each word as a **vector** in a high-dimensional space.

- **Similar meanings ⇒ nearby vectors**.
- Enables graded similarity (not just "same" or "different").
- Allows generalisation to *unseen but similar* words.

Why vectors help ML:

- One-hot features require exact word match between train and test.
- Embedding features: similar words have similar vectors ⇒ classifier generalises.

2.2.4 Co-occurrence Matrices

Term-Document Matrix

Rows = vocabulary words; Columns = documents.

Cell (w, d) = count (or weighted count) of word w in document d.

Use case: Information retrieval, document similarity.

Word-Word (Term-Term) Matrix

Rows = vocabulary words; Columns = also vocabulary words.

Cell (w, c) = count of context word c within a $\pm k$ window of target word w.

Use case: Capturing fine-grained semantic similarity.

	Term-Document	**Word-Word**
Columns	Documents	Context words
Window	Whole document	$\pm k$ words (e.g., ± 4)
Typical use	IR, topic models	Word similarity

Example: Mini Word-Word Matrix

Corpus windows:

- "...cherry **pie**, a traditional ..."
- "...strawberry rhubarb **pie**. Apple **pie** ..."
- "...personal **digital** assistants ..."
- "...**information** available on the internet ..."

	`a`	`computer`	`pie`
`cherry`	1	0	1
`strawberry`	0	0	2
`digital`	0	1	0
`information`	1	1	0

Observation: `cherry` and `strawberry` share the `pie` dimension; `digital` and `information` share `computer`.

Sparsity

Full word-word matrices are $|V| \times |V|$ (e.g., $50{,}000 \times 50{,}000$). Most entries are zero $\Rightarrow$ **sparse vectors**. Efficient sparse-matrix algorithms are essential.

2.2.5 Similarity Metrics: Dot Product and Cosine

Dot Product

$$\vec{v} \cdot \vec{w} = \sum_{i=1}^{N} v_i w_i$$

High when vectors have large values in the same dimensions.

Problem: Favours *long* vectors $\Rightarrow$ frequent words dominate.

Cosine Similarity

$$\cos(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}|\,|\vec{w}|} = \frac{\sum_i v_i w_i}{\sqrt{\sum_i v_i^2}\,\sqrt{\sum_i w_i^2}}$$

Properties:

- Normalises by vector length $\Rightarrow$ removes frequency bias.
- Focuses on *direction* (pattern of co-occurrences), not magnitude.
- For non-negative vectors: ranges from 0 (orthogonal) to 1 (identical direction).

Example: Cosine Calculation

Given (from Wikipedia):

	`pie`	`data`	`computer`
`cherry`	442	8	2
`digital`	5	1683	1670
`information`	5	3982	3325

$$\cos(\text{cherry}, \text{information}) = \frac{442 \cdot 5 + 8 \cdot 3982 + 2 \cdot 3325}{\sqrt{442^2 + 8^2 + 2^2}\,\sqrt{5^2 + 3982^2 + 3325^2}} \approx 0.017$$

$$\cos(\text{digital}, \text{information}) \approx 0.996$$

Conclusion: `information` is much closer to `digital` than to `cherry`.

2.2.6 Weighting: tf-idf

Raw counts overweight frequent but uninformative words (e.g., `the`, `good`).

tf-idf

$$\text{tf-idf}(t, d) = \text{tf}_{t,d} \times \text{idf}_t$$

Term frequency (tf):

$$\mathrm{tf}_{t,d} = \begin{cases} 1 + \log_{10}(\mathrm{count}(t,d)) & \text{if count} > 0 \\ 0 & \text{otherwise} \end{cases}$$

Inverse document frequency (idf):

$$\mathrm{idf}_t = \log_{10}\left(\frac{N}{\mathrm{df}_t}\right)$$

where N = total documents, df_t = number of documents containing t.

Example: idf in Shakespeare's 37 Plays

Word	df	idf	Interpretation
`Romeo`	1	1.57	Very discriminative (one play)
`Falstaff`	4	0.97	Moderately discriminative
`good`	37	0	Appears everywhere ⇒ no discriminative power

Key insight: If $\mathrm{df}_t = N$, then $\mathrm{idf}_t = \log(1) = 0$, so tf-idf = 0.

What Counts as a "Document"?

For tf-idf, a "document" can be a play, article, paragraph, or even a sentence — whatever granularity suits the task.

2.2.7 Dense Embeddings: Word2Vec

Sparse vs. Dense Vectors

	Sparse (count-based)	Dense (learned)
Dimensionality	$\lvert V \rvert$ (20k–50k)	50–1000
Most entries	Zero	Non-zero
Synonymy	Distinct dimensions	Similar vectors
Generalisation	Limited	Better

Word2Vec: Skip-Gram with Negative Sampling (SGNS)

Idea: Learn embeddings by training a classifier to predict whether a word c appears near target w.

Self-supervision: Positive examples come from actual context; no manual labels needed.

Training procedure:

1. For each target word w in the corpus, collect context words $c_1, \ldots, c_L$ within window $\pm k$.
2. These (w, c_i) pairs are **positive examples**.
3. Randomly sample k **negative examples** (w, c_{neg}) from the vocabulary.
4. Train a logistic classifier to distinguish positive from negative pairs.
5. The learned weight matrices become the embeddings.

Skip-Gram Probability Model

Probability that c is a true context word of w:

$$P(+|w,c) = \sigma(c \cdot w) = \frac{1}{1+\exp(-c \cdot w)}$$

Sigmoid converts the dot product into a probability.

For all context words (assuming independence):

$$P(+|w, c_{1:L}) = \prod_{i=1}^{L} \sigma(c_i \cdot w)$$

Skip-Gram Loss Function (Negative Sampling)

For one target w with positive context c_{pos} and k negative samples $c_{\text{neg}_1}, \ldots, c_{\text{neg}_k}$:

$$\mathcal{L} = -\left[\log \sigma(c_{\text{pos}} \cdot w) + \sum_{i=1}^{k} \log \sigma(-c_{\text{neg}_i} \cdot w)\right]$$

Goal: Maximise dot product with true context; minimise dot product with noise.

Why Negative Sampling?

Full softmax requires normalising over all $|V|$ words at each step — computationally prohibitive. Negative sampling approximates this by contrasting against a small random sample.

Example: Skip-Gram Training Data

Sentence: "…lemon, a **tablespoon of apricot jam, a** pinch …"

Target: `apricot`, window ± 2

Positive pairs: (`apricot`, `tablespoon`), (`apricot`, `of`), (`apricot`, `jam`), (`apricot`, `a`)

Negative pairs (sampled): (`apricot`, `aardvark`), (`apricot`, `matrix`), …

Gradient descent update intuition:

- Move w and c_{pos} closer (increase their dot product).
- Move w and each c_{neg} apart (decrease their dot products).

Two Embedding Matrices

Skip-Gram learns:

- W — target word embeddings
- C — context word embeddings

Common practice: combine them as $\vec{w}_i + \vec{c}_i$ for the final representation.

2.2.8 Properties of Embeddings

Window size effects:

Window	Nearest Neighbours
Small (± 2)	Syntactically/taxonomically similar (e.g., `Hogwarts` $\rightarrow$ other fictional schools)
Large (± 5)	Topically related (e.g., `Hogwarts` $\rightarrow$ `Dumbledore`, `Malfoy`)

Analogical reasoning (parallelogram method):

Parallelogram Model for Analogies

To solve "a is to b as a^* is to ?":

$$\hat{b}^* = \arg\max_x \text{ similarity}(x,\ a^* - a + b)$$

Examples:

- $\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$
- $\vec{\text{Paris}} - \vec{\text{France}} + \vec{\text{Italy}} \approx \vec{\text{Rome}}$

Caveats with Analogies

- Works mainly for **frequent words** and **certain relation types** (capitals, POS).
- Fails for rarer words, longer distances, and many semantic relations.
- Nearest neighbours may include query words themselves.

Bias in embeddings:

Embeddings Reflect Corpus Biases

- "father : doctor :: mother : ?" $\rightarrow$ `nurse`
- "man : computer programmer :: woman : ?" $\rightarrow$ `homemaker`

Implication: Systems using biased embeddings (e.g., for hiring) can perpetuate discrimination.

Historical embeddings trained on different decades reveal changing cultural biases (e.g., gender stereotypes declining 1960–1990).

2.2.9 Summary: Sparse vs. Dense Methods

	Count-Based (Sparse)	Word2Vec (Dense)
Representation	Co-occurrence counts (tf-idf, PMI)	Learned weight vectors
Dimensionality	$\lvert V \rvert$	50–1000
Training	Direct matrix construction	SGD on prediction task
Synonymy	Separate dimensions	Similar vectors
Interpretability	High (dimensions = words)	Low (abstract dimensions)
Generalisation	Limited	Better (nearby = similar)

3 Week 3: Statistical Language Models

3.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Define a **language model** and use the **chain rule** with the **Markov assumption** to build N-gram approximations.
2. Estimate N-gram probabilities with **maximum likelihood estimation (MLE)** from corpus counts.
3. Handle sparsity via **smoothing**, **interpolation**, and **backoff**.
4. Evaluate language models using **training/dev/test sets** and **perplexity**.
5. Explain strengths and limitations of N-gram models, including their inability to capture **long-distance dependencies**.

The Big Picture

This week answers: *"How can we predict the next word in a sequence?"*

N-gram language models assign probabilities to word sequences by:

- Decomposing $P(w_1, \ldots, w_n)$ using the **chain rule**.
- Approximating long histories with the **Markov assumption** (only last $N - 1$ words matter).
- Estimating probabilities from **corpus counts** (MLE).

Why study N-grams? They introduce foundational concepts (perplexity, smoothing, train/test splits) that carry over to modern LLMs.

What Examiners Like to Ask

- Define a language model; write the chain rule decomposition.
- State the Markov assumption for bigrams/trigrams.
- Compute bigram probabilities from counts (MLE).
- Explain why zeros occur and how **add-one smoothing** addresses them.
- Define **perplexity**; explain why lower is better.
- Compare **interpolation** vs. **backoff**.
- Explain why N-grams fail on long-distance dependencies.

3.2 Overview + Core Concepts + Extended Theory

Roadmap

1. What is a language model?
2. Chain rule and Markov assumption
3. N-gram models: unigram, bigram, trigram
4. Maximum likelihood estimation (MLE)
5. Practical issues: log probabilities, vocabulary

6. Evaluation: train/dev/test sets and perplexity
7. Sampling from N-gram models
8. Sparsity and zeros
9. Smoothing: add-one (Laplace)
10. Interpolation and backoff
11. Limitations of N-gram models

3.2.1 What is a Language Model?

Language Model (LM)

A **language model** assigns probabilities to sequences of words.

Two equivalent views:

1. Probability of a whole sentence: $P(w_1, w_2, \ldots, w_n)$
2. Probability of the next word given history: $P(w_n|w_1, \ldots, w_{n-1})$

Applications:

- **Spell/grammar checking:** "Their are two midterms" → "There are two midterms"
- **Speech recognition:** "I will be back soonish" vs. "I will be bassoon dish"
- **Text generation:** LLMs predict words left-to-right (autoregressive).

3.2.2 The Chain Rule of Probability

Chain Rule

For any sequence of random variables:

$$P(x_1, x_2, \ldots, x_n) = \prod_{k=1}^{n} P(x_k|x_1, \ldots, x_{k-1})$$

Applied to words:

$$P(w_1, \ldots, w_n) = P(w_1) \cdot P(w_2|w_1) \cdot P(w_3|w_1, w_2) \cdots P(w_n|w_1, \ldots, w_{n-1})$$

Example: Chain Rule

$P($"The water of Walden Pond"$)$

$$= P(\text{The}) \times P(\text{water}|\text{The}) \times P(\text{of}|\text{The water}) \times \cdots$$

Problem: We can't estimate $P(w_n|w_1, \ldots, w_{n-1})$ directly — there are too many possible histories!

3.2.3 The Markov Assumption

Markov Assumption

Approximate the full history by only the last $N-1$ words:

$$P(w_n|w_1,\ldots,w_{n-1}) \approx P(w_n|w_{n-N+1},\ldots,w_{n-1})$$

Intuition: Recent context is most predictive; distant history matters less.

3.2.4 N-gram Models

N-gram Model

An **N-gram model** conditions on the previous $N-1$ words.

Model	N	Approximation
Unigram	1	$P(w_n)$ — no context
Bigram	2	$P(w_n\|w_{n-1})$ — 1 word of context
Trigram	3	$P(w_n\|w_{n-2},w_{n-1})$ — 2 words of context
4-gram	4	$P(w_n\|w_{n-3},w_{n-2},w_{n-1})$

Example: Bigram Approximation

Instead of:

$$P(\text{blue}|\text{The water of Walden Pond is so beautifully})$$

Approximate with:

$$P(\text{blue}|\text{beautifully})$$

Example: Shakespeare Generation by N-gram Order

Unigram: "To him swallowed confess hear both. Which. Of save on trail for are ay device…"

Bigram: "Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry."

Trigram: "Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done."

4-gram: "King Henry. What! I will go seek the traitor Gloucester."

Observation: Higher N ⇒ more coherent text (but more memorisation of training data).

3.2.5 Maximum Likelihood Estimation (MLE)

MLE for Bigrams

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1},w_n)}{C(w_{n-1})}$$

where $C(\cdot)$ denotes corpus count.

General N-gram:

$$P(w_n|w_{n-N+1}^{n-1}) = \frac{C(w_{n-N+1}^{n})}{C(w_{n-N+1}^{n-1})}$$

Example: Bigram Counts (Berkeley Restaurant Project)

From 9,222 sentences:

	`want`	`to`	`eat`	`chinese`
`i`	827	0	9	0
`want`	0	608	1	6
`to`	0	4	686	2

Given unigram counts: $C(\text{i}) = 2533$, $C(\text{want}) = 927$, $C(\text{to}) = 2417$

$$P(\text{want}|\text{i}) = \frac{827}{2533} = 0.33, \quad P(\text{to}|\text{want}) = \frac{608}{927} = 0.66$$

Example: Sentence Probability

$P(\text{¡s¿ i want english food ¡/s¿})$

$$= P(\text{i}|\text{¡s¿}) \times P(\text{want}|\text{i}) \times P(\text{english}|\text{want}) \times P(\text{food}|\text{english}) \times P(\text{¡/s¿}|\text{food})$$

$$= 0.25 \times 0.33 \times 0.0011 \times 0.5 \times 0.68 \approx 0.000031$$

What N-grams Capture

- **Grammar:** $P(\text{to}|\text{want}) = 0.66$ (want $\rightarrow$ infinitive)
- **World knowledge:** $P(\text{chinese}|\text{want}) > P(\text{english}|\text{want})$
- **Semantics:** "dinner" and "lunch" have similar distributions

---

3.2.6 Practical Issues: Log Probabilities

Log Probabilities

Multiplying many small probabilities $\Rightarrow$ **underflow**.

Solution: Work in log space.

$$\log(p_1 \times p_2 \times \cdots \times p_n) = \log p_1 + \log p_2 + \cdots + \log p_n$$

Convert back at the end: $p = \exp(\text{logprob})$

3.2.7 Sentence Boundary Tokens

Start and End Tokens

Insert special tokens to model sentence boundaries:

- `<s>` — start of sentence
- `</s>` — end of sentence

This allows the model to learn $P(w_1|¡s¿)$ and $P(¡/s¿|w_n)$.

3.2.8 Evaluation: Train / Dev / Test Sets

Data Splits

Set	Purpose
Training	Estimate model parameters (N-gram counts)
Development (Dev)	Tune hyperparameters (e.g., interpolation λs); can evaluate repeatedly
Test	Final evaluation *only once* (or very few times)

Training on the Test Set = Bad Science!

If test sentences influence training:

- Model assigns artificially high probability to test data.
- Evaluation metric is falsely optimistic.
- Results don't generalise.

3.2.9 Perplexity

Perplexity (PP)

$$\mathrm{PP}(W) = P(w_1, w_2, \ldots, w_N)^{-1/N} = \sqrt[N]{\frac{1}{P(w_1, \ldots, w_N)}}$$

Interpretation:

- Inverse probability, normalised by sequence length.
- **Weighted average branching factor:** how many equally likely words could follow on average.
- **Lower perplexity = better model.**

Range: $[1, \infty)$ (probability in $[0, 1]$).

Example: Perplexity Calculation

Deterministic language $L = \{\text{red}, \text{blue}, \text{green}\}$, each word equally likely ($p = 1/3$).

Test set T = "red red red red blue" (5 words).

$$P(T) = (1/3)^5, \quad \text{PP}(T) = ((1/3)^5)^{-1/5} = (1/3)^{-1} = 3$$

Now suppose training made `red` very likely: $P(\text{red}) = 0.8$, $P(\text{blue}) = 0.1$.

$$P(T) = 0.8^4 \times 0.1 = 0.04096, \quad \text{PP}(T) = 0.04096^{-1/5} \approx 1.89$$

Lower perplexity $\Rightarrow$ model better predicts the test set.

Example: Perplexity by N-gram Order (WSJ)

Training: 38M words; Test: 1.5M words.

	Unigram	Bigram	Trigram
Perplexity	962	170	109

Observation: More context $\Rightarrow$ lower perplexity.

Why Log Perplexity?

Log perplexity turns products into sums, making computation **numerically stable**.

3.2.10 Sampling from N-gram Models

Shannon Sampling Method

1. Sample w_1 from $P(w|$¡s¿$)$.
2. Sample w_2 from $P(w|w_1)$ (for bigram).
3. Repeat until `</s>` is sampled.
4. Concatenate the words.

Use: Visualise what the model has learned; diagnose overfitting.

3.2.11 Sparsity and Zeros

The Sparsity Problem

Most possible N-grams never appear in training.
Shakespeare corpus: $N = 884{,}647$ tokens, $V = 29{,}066$ types.

- Possible bigrams: $V^2 \approx 844$ million.
- Observed bigram types: $\sim 300{,}000$ (only 0.04%).

Consequence: Unseen N-grams get $P = 0$ under MLE.

Why Zeros Are Fatal

- If any N-gram in test set has $P = 0$, the whole sentence has $P = 0$.
- Perplexity $= \infty$ (can't divide by 0).

Example: Zero Probability

Training: "ate lunch", "ate dinner", "ate a", "ate the"

Test: "ate **breakfast**"

$\Rightarrow P(\text{breakfast}|\text{ate}) = 0$ under MLE.

3.2.12 Smoothing: Add-One (Laplace)

Add-One Smoothing

Pretend every N-gram was seen one extra time.

MLE:

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}, w_n)}{C(w_{n-1})}$$

Add-one (Laplace):

$$P_{\text{Laplace}}(w_n|w_{n-1}) = \frac{C(w_{n-1}, w_n) + 1}{C(w_{n-1}) + V}$$

where V = vocabulary size.

Example: Add-One Smoothing

Raw counts:

	want	to	eat
i	827	0	9

With $C(\text{i}) = 2533$ and $V = 1446$:

$$P_{\text{Laplace}}(\text{to}|\text{i}) = \frac{0 + 1}{2533 + 1446} = \frac{1}{3979} \approx 0.00025$$

Previously zero $\Rightarrow$ now non-zero!

Problem with Add-One

Add-one is a **blunt instrument**:

- Shifts too much probability mass from frequent to rare/unseen events.
- Reconstituted counts show dramatic changes (e.g., $827 \to 527$).
- Generally **not used** for N-grams in practice.

When it's used: Naive Bayes text classification, domains with fewer zeros.

3.2.13 Interpolation and Backoff

Interpolation

Combine estimates from different N-gram orders:

$$\hat{P}(w_n|w_{n-2}, w_{n-1}) = \lambda_1 P(w_n|w_{n-2}, w_{n-1}) + \lambda_2 P(w_n|w_{n-1}) + \lambda_3 P(w_n)$$

where $\lambda_1 + \lambda_2 + \lambda_3 = 1$.

Key idea: If trigram evidence is sparse, lean more on bigram/unigram.

Backoff

Use the highest-order N-gram with sufficient evidence:

1. If $C(w_{n-2}, w_{n-1}, w_n) > 0$, use trigram.
2. Else if $C(w_{n-1}, w_n) > 0$, use bigram.
3. Else use unigram.

Complication: Must discount higher-order N-grams so probabilities sum to 1.

Stupid Backoff

Simplified backoff without proper discounting (not a true probability distribution):

$$S(w_i|w_{i-k+1}^{i-1}) = \begin{cases} \dfrac{C(w_{i-k+1}^{i})}{C(w_{i-k+1}^{i-1})} & \text{if } C(w_{i-k+1}^{i}) > 0 \\ 0.4 \cdot S(w_i|w_{i-k+2}^{i-1}) & \text{otherwise} \end{cases}$$

Works well at web scale (e.g., Google N-grams).

How to Set λs?

Tune on a **held-out (dev) set**:

1. Fix N-gram probabilities from training data.
2. Search for λs that minimise perplexity on dev set.

Interpolation vs. Backoff

Interpolation usually works better because it always uses information from all orders, not just the highest available.

3.2.14 Limitations of N-gram Models

Why N-grams Fail

1. **Long-distance dependencies:**
   "The soups that I made from that new cookbook I bought yesterday **were** amazingly delicious."
   Trigram can't see that "soups" (plural) requires "were" — too far away.
2. **No generalisation across similar meanings:**
   N-grams treat "car" and "automobile" as completely unrelated dimensions.

3. **Sparsity grows exponentially with N.**

Solution: Large language models (LLMs) with embeddings and attention.

But N-grams are still useful:

- Fast and lightweight.
- Good for constrained domains.
- Introduce key concepts: perplexity, smoothing, train/test, sampling.

3.2.15 Summary: N-gram Language Models

Concept	**Key Point**
Language model	Assigns probability to word sequences
Chain rule	$P(w_1, \ldots, w_n) = \prod_k P(w_k \mid w_1, \ldots, w_{k-1})$
Markov assumption	Approximate history with last $N-1$ words
MLE	$P(w_n \mid w_{n-1}) = C(w_{n-1}, w_n)/C(w_{n-1})$
Perplexity	PP $= P(W)^{-1/N}$; lower = better
Add-one smoothing	Add 1 to all counts; blunt but simple
Interpolation	Weighted mix of unigram, bigram, trigram
Backoff	Fall back to lower-order N-gram if higher is zero
Limitation	Can't capture long-distance dependencies

4 Week 4: Neural Language Models, RNNs, and LSTMs

4.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Explain **embeddings** and the **embedding matrix**, including how token indices (or one-hot vectors) are mapped to embedding vectors.
2. Use embeddings as input to **neural classifiers**, and combine them with methods such as **pooling** to produce sentence-level representations.
3. Describe simple **feedforward neural language models**, which use token embeddings in a fixed window to predict the next word.
4. Compare neural LMs with n-gram models, highlighting advantages such as **generalisation** and longer context.
5. Explain **recurrent neural networks (RNNs)** and how they process sequences with hidden states.
6. Describe how RNNs are trained, including **backpropagation through time** and **teacher forcing**.
7. Apply RNNs to language modelling and NLP tasks, such as sequence labelling and text generation.
8. Motivate and explain **LSTMs**, including their **gating mechanisms** for handling long-distance dependencies.

The Big Picture

This week transitions from count-based and simple neural methods to **sequence-aware architectures**. We start with embeddings as learned representations fed into feedforward classifiers and language models, then introduce **RNNs** which maintain hidden states across time steps, enabling modelling of sequences of arbitrary length. Finally, we cover **LSTMs**, which solve the vanishing gradient problem through gating mechanisms, enabling capture of long-distance dependencies.

What Examiners Like to Ask

- Explain how the **embedding matrix** E maps one-hot vectors (or indices) to dense embeddings.
- Describe **pooling** (mean pooling) for converting variable-length sequences into fixed-size representations.
- Compare **feedforward neural LMs** to **n-gram LMs**: context size, generalisation, sparsity handling.
- Write the **RNN forward equations** ($h_t = g(Uh_{t-1} + Wx_t)$, $y_t = f(Vh_t)$).
- Explain **unrolling** for training and why RNNs require incremental inference.
- Define **teacher forcing** and **weight tying** in RNN language model training.
- Describe the **vanishing gradient problem** and why it motivates LSTMs.
- Explain the **LSTM gating mechanisms**: forget gate, input/add gate, output gate.
- Describe the **encoder-decoder architecture** and its components.
- Explain **attention** as a solution to the context bottleneck in encoder-decoder models.

4.2 Overview + Core Concepts + Extended Theory

Roadmap

1. Embeddings and the embedding matrix E
2. Embeddings as input to feedforward classifiers (pooling)
3. Feedforward neural language models
4. Neural LMs vs. n-gram models
5. Recurrent Neural Networks (RNNs / Elman Nets)
6. Training RNNs: unrolling, backpropagation through time, teacher forcing
7. RNNs for NLP tasks: LM, sequence labelling, classification, generation
8. LSTMs: gating mechanisms for long-distance dependencies
9. Encoder-decoder architecture
10. Attention mechanism

4.2.1 Embeddings and the Embedding Matrix

Embedding Matrix E

An **embedding** is a dense vector of dimension d representing a token.

The **embedding matrix** E is a learned lookup table:

- Shape: $|V| \times d$ (one row per vocabulary token)
- Each row $E[i]$ is the embedding for token i

Embedding matrices are **central to all modern NLP systems**.

One-Hot to Embedding Conversion

Method 1: Indexing (efficient)

- Given token index i, select row $E[i]$ directly.

Method 2: Matrix multiplication (conceptual)

- Represent token as one-hot vector $\vec{x} \in \mathbb{R}^{|V|}$ where $x_i = 1$, all others 0.
- Multiply: $\vec{e} = \vec{x}^\top E$ selects row i.

Both are equivalent; indexing is faster in practice.

Example: Selecting Embeddings

Vocabulary: $|V| = 10000$, embedding dimension $d = 300$

Token: "dessert" has index $i = 3$

One-hot vector: $\vec{x} = [0, 0, 1, 0, \ldots, 0]$ (1 at position 3)

Embedding: $\vec{e}_{\text{dessert}} = E[3] \in \mathbb{R}^{300}$

For N tokens, we get N embeddings, forming an $N \times d$ matrix.

4.2.2 Embeddings as Input to Feedforward Classifiers

Representation Learning

Instead of hand-crafted features (like "count of positive lexicon words"), we use **learned embeddings** as input features.

Benefit: The network learns useful representations from data.

Problem: Text comes in variable lengths, but classifiers need fixed-size input.

Solutions:

Method	Description
Concatenation	Concatenate all N embeddings into one vector of shape $[1 \times Nd]$. Pad shorter sequences; truncate longer ones.
Pooling	Combine N embeddings into a single $[1 \times d]$ vector (same dimensionality as one word).

Mean Pooling

Given N token embeddings $e(w_1), \ldots, e(w_N)$, compute the element-wise mean:

$$\vec{x}_{\text{mean}} = \frac{1}{N} \sum_{i=1}^{N} e(w_i)$$

Intuition: For sentiment analysis, exact word position matters less than overall word content.

Equations for pooled classifier:

$$\begin{aligned}
\vec{x} &= \text{mean}(e(w_1), \ldots, e(w_N)) \\
\vec{h} &= \sigma(W\vec{x} + \vec{b}) \\
\vec{z} &= U\vec{h} \\
\hat{\vec{y}} &= \text{softmax}(\vec{z})
\end{aligned}$$

Pooling vs. Concatenation Trade-off

- **Pooling:** Less information (loses position), but efficient and handles variable length naturally.
- **Concatenation:** Preserves position, but requires padding/truncation and more parameters.

4.2.3 Feedforward Neural Language Models

Feedforward Neural LM

Task: Predict next word w_t given prior words $w_{t-1}, w_{t-2}, \ldots$

Approach: Use a **sliding window** of fixed size $N - 1$ (like n-grams).

Architecture:

1. Look up embeddings for context words: $e_{t-1}, e_{t-2}, \ldots, e_{t-N+1}$

2. Concatenate into input vector: $\vec{e} = [e_{t-N+1}; \ldots; e_{t-1}]$
3. Pass through hidden layer: $\vec{h} = \sigma(W\vec{e} + \vec{b})$
4. Output layer with softmax over vocabulary: $\hat{\vec{y}} = \text{softmax}(U\vec{h})$

Example: 4-gram Feedforward LM

Context: "and thanks for all the __"

Window ($N = 4$): ["for", "all", "the"] → predict "fish"

Dimensions:

- Embedding layer: $[1 \times Nd]$ (concatenated embeddings)
- Hidden layer: $[1 \times d_h]$
- Output layer: $[1 \times |V|]$ (probability over vocabulary)

$\hat{y}_{42}$ gives $P(w_t = \text{"fish"} | w_{t-3}, w_{t-2}, w_{t-1})$

4.2.4 Neural LMs vs. N-gram LMs

Why Neural LMs Outperform N-gram LMs

Generalisation through embeddings:

Training data: "I have to make sure that the **cat** gets fed."

Test data: "I forgot to make sure that the **dog** gets __"

- **N-gram LM:** Never saw "dog gets"; cannot predict "fed".
- **Neural LM:** Uses similarity of "cat" and "dog" embeddings to generalise and predict "fed".

	N-gram LM	**Neural LM**
Representation	Discrete word identities	Continuous embeddings
Generalisation	None (exact match only)	Similar words → similar predictions
Sparsity	Severe (many zero counts)	No zeros (dense vectors)
Context	Fixed $N - 1$ words	Fixed window (feedforward)
Parameters	$O(\|V\|^N)$	$O(\|V\| \cdot d + d \cdot d_h)$

4.2.5 Recurrent Neural Networks (RNNs)

Recurrent Neural Network (RNN)

A **recurrent neural network** is any network containing a **cycle** in its connections.

The hidden state h_t depends on:

- The current input x_t
- The previous hidden state h_{t-1}

This allows information to **persist across time steps**.

Simple RNN (Elman Net) Equations

Forward inference:

$$h_t = g(Uh_{t-1} + Wx_t)$$
$$y_t = f(Vh_t)$$

where:

- x_t: input at time t (e.g., embedding)
- h_t: hidden state at time t
- y_t: output at time t
- W: input-to-hidden weights $[d_h \times d]$
- U: hidden-to-hidden weights $[d_h \times d_h]$
- V: hidden-to-output weights $[|V| \times d_h]$
- g: activation function (e.g., tanh, ReLU)
- f: output function (e.g., softmax)

Key Property: Sequential Inference

Computing h_t **requires** h_{t-1}.

$\Rightarrow$ Inference must proceed **incrementally** from start to end.

$\Rightarrow$ Cannot parallelise across time steps (unlike feedforward networks).

RNN Forward Inference Algorithm

```
function FORWARD_RNN(x, network):
    h_0 = 0  # initial hidden state
    for i = 1 to LENGTH(x):
        h_i = g(U * h_{i-1} + W * x_i)
        y_i = f(V * h_i)
    return y
```

Note: Matrices U, V, W are **shared across all time steps**.

4.2.6 RNNs as Language Models

RNN LM vs. Feedforward LM

	Feedforward LM	**RNN LM**
Context	Fixed window size	Entire preceding history (via h_{t-1})
Representation	Concatenated embeddings	Hidden state encodes history

RNN Language Model Equations

$$
\begin{aligned}
e_t &= Ex_t && \text{(embedding lookup)} \\
h_t &= g(Uh_{t-1} + We_t) && \text{(update hidden state)} \\
\hat{y}_t &= \text{softmax}(Vh_t) && \text{(predict next word)}
\end{aligned}
$$

Probability of next word being k:

$$P(w_{t+1} = k \mid w_1, \ldots, w_t) = \hat{y}_t[k]$$

Probability of entire sequence:

$$P(w_{1:n}) = \prod_{i=1}^{n} P(w_i \mid w_{<i}) = \prod_{i=1}^{n} \hat{y}_{i-1}[w_i]$$

4.2.7 Training RNNs

Unrolling Through Time

To train an RNN, we **unroll** it into a feedforward computational graph:

1. Given input sequence, create copies of the network for each time step.
2. Weight matrices (U, V, W) are **shared** across all copies.
3. Apply standard backpropagation to the unrolled graph.

This is called **Backpropagation Through Time (BPTT)**.

Self-Supervision for LM Training

Self-supervision: The text itself provides the training signal.

- At each time step t, the model predicts w_{t+1}.
- The true next word w_{t+1} serves as the gold label.
- No manual annotation required.

Loss function: Cross-entropy at each position.

$$L_{CE}(\hat{y}_t, y_t) = -\log \hat{y}_t[w_{t+1}]$$

Since y_t is one-hot, only the probability of the correct word matters.

Teacher Forcing

Teacher forcing: During training, always feed the **correct (gold) previous word** as input, not the model's prediction.

Why?

- Early in training, predictions are poor.
- Feeding wrong predictions would compound errors.
- Gold context ensures stable training signal.

At test time: We must use the model's own predictions (no gold available).

Weight Tying

The embedding matrix E and output matrix V serve similar roles:

- E: maps words to embeddings (rows are word vectors)
- V: maps hidden states to word scores (rows are word vectors)

Weight tying: Use $V = E^\top$ instead of learning separate matrices.
Weight-tied RNN LM equations:

$$\begin{aligned} e_t &= Ex_t \\ h_t &= g(Uh_{t-1} + We_t) \\ \hat{y}_t &= \text{softmax}(E^\top h_t) \end{aligned}$$

Benefits: Fewer parameters, often better generalisation.

4.2.8 RNNs for NLP Tasks

Four main architectures:

Task Type	Example	Architecture
Sequence Labelling	POS tagging, NER	One output per input token
Sequence Classification	Sentiment analysis	One output for entire sequence (use final h_n or pooling)
Language Modelling	Next word prediction	Output at each step predicts next token
Encoder-Decoder	Translation, Summarisation	Separate encoder and decoder RNNs

Stacked RNNs

Multiple RNN layers stacked on top of each other:

- Output of layer 1 becomes input to layer 2.
- Captures more abstract representations at higher layers.

Bidirectional RNNs

Run **two separate RNNs**:

- **Forward RNN:** processes $x_1, \ldots, x_n$ (left-to-right)
- **Backward RNN:** processes $x_n, \ldots, x_1$ (right-to-left)

Concatenate their hidden states:

$$h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t]$$

Benefit: Each position has context from **both** left and right.
Use case: Sequence labelling, classification (but **not** language modelling, which must be left-to-right).

Autoregressive Generation

To generate text:

1. Start with `<s>` token.
2. Predict distribution over next word.
3. **Sample** from distribution (or take argmax for greedy decoding).
4. Feed sampled word as next input.
5. Repeat until `</s>` or max length.

4.2.9 Long Short-Term Memory Networks (LSTMs)

Motivation: Problems with Vanilla RNNs

Problem 1: Long-distance dependencies

- Example: "The flights the airline was canceling **were** full."
- "were" depends on "flights" (far away), not "canceling" (nearby).
- Hidden state must carry information over many steps.

Problem 2: Vanishing gradients

- During backpropagation, gradients are multiplied through many time steps.
- Gradients can shrink exponentially, preventing learning of long-range dependencies.

LSTM Key Idea

LSTMs divide context management into two subproblems:

1. **Removing** information no longer needed
2. **Adding** information likely needed for future decisions

LSTMs introduce:

- An explicit **context/cell state** c_t (separate from hidden state h_t)
- **Gates**: neural circuits that control information flow

LSTM Gates

1. Forget Gate — decides what to **remove** from context:

$$
\begin{aligned}
f_t &= \sigma(U_f h_{t-1} + W_f x_t) \\
k_t &= c_{t-1} \odot f_t
\end{aligned}
$$

2. Input/Add Gate — decides what to **add** to context:

$$
\begin{aligned}
g_t &= \tanh(U_g h_{t-1} + W_g x_t) && \text{(candidate values)} \\
i_t &= \sigma(U_i h_{t-1} + W_i x_t) && \text{(gate)} \\
j_t &= g_t \odot i_t \\
c_t &= j_t + k_t && \text{(new context)}
\end{aligned}
$$

3. Output Gate — decides what to **output** as hidden state:

$$
\begin{aligned}
o_t &= \sigma(U_o h_{t-1} + W_o x_t) \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}
$$

where $\odot$ denotes element-wise (Hadamard) multiplication.

Intuition for LSTM Gates

- **Forget gate f_t:** Values near 0 erase; values near 1 preserve.
- **Input gate i_t:** Values near 0 block new info; values near 1 allow it.
- **Output gate o_t:** Controls how much of cell state reaches hidden state.

The cell state c_t acts as a **conveyor belt** carrying information across time with minimal modification (additive updates, not multiplicative).

4.2.10 Encoder-Decoder Architecture

Encoder-Decoder Model

For sequence-to-sequence tasks (translation, summarisation):

Three components:

1. **Encoder:** Reads input sequence $x_{1:n}$, produces hidden states $h^e_{1:n}$.
2. **Context vector:** $c = h^e_n$ (final encoder hidden state).
3. **Decoder:** Takes c as initial state, generates output sequence $y_{1:m}$.

Encoder-Decoder Equations

Encoder:

$$h^e_t = g(h^e_{t-1}, x_t)$$

Context:

$$c = h^e_n$$

Decoder:

$$\begin{aligned} h^d_0 &= c \\ h^d_t &= g(\hat{y}_{t-1}, h^d_{t-1}, c) \\ \hat{y}_t &= \text{softmax}(h^d_t) \end{aligned}$$

Conditional probability:

$$P(y \mid x) = \prod_{t=1}^{m} P(y_t \mid y_{1:t-1}, x)$$

Bottleneck Problem

The entire source sequence must pass through **one fixed-size vector** c.

For long sequences, this is a severe **information bottleneck**.

Solution: Attention mechanism.

4.2.11 Attention Mechanism

Attention: Key Idea

Instead of using only the final encoder state:

- Compute a **weighted average** of **all** encoder hidden states.
- Weights depend on **relevance** to the current decoder state.
- Different decoder steps attend to different parts of the input.

Attention Computation

At decoder time step i:

1. Compute attention scores (dot-product attention):

$$\text{score}(h^d_{i-1}, h^e_j) = h^d_{i-1} \cdot h^e_j$$

2. Normalise with softmax to get weights:

$$\alpha_{ij} = \frac{\exp(\text{score}(h^d_{i-1}, h^e_j))}{\sum_k \exp(\text{score}(h^d_{i-1}, h^e_k))}$$

3. Compute context vector as weighted sum:

$$c_i = \sum_j \alpha_{ij} h^e_j$$

4. Use c_i in decoder:

$$h^d_i = g(\hat{y}_{i-1}, h^d_{i-1}, c_i)$$

Attention Interpretation

- α_{ij}: how much decoder step i "attends to" encoder position j.
- High α_{ij}: encoder position j is relevant for predicting output i.
- Attention weights are often visualised as alignment matrices.

Example: Translation with Attention

Source (English): "The green witch arrived"

Target (Spanish): "Llegó la bruja verde"

When generating "bruja" (witch):

- High attention weight on "witch"
- Lower weights on "The", "green", "arrived"

The model learns soft word alignments automatically!

4.2.12 Summary: Neural Architectures for Sequences

Architecture	**Context**	**Key Feature**	**Use Cases**
Feedforward NN	Fixed window	Simple, parallelisable	Classification, fixed-context LM
Simple RNN	Unbounded (via h_{t-1})	Sequential processing	LM, sequence tasks
LSTM	Unbounded + gated	Handles long-range dependencies	LM, translation, tagging
Encoder-Decoder	Full source sequence	Seq-to-seq mapping	Translation, summarisation
+ Attention	Dynamic focus	Soft alignment	Improved translation

5 Week 5: Transformer Models and Large Language Models

5.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Define **Large Language Models (LLMs)** and explain how they extend earlier language models (n-gram, RNN) through scale and architecture.
2. Distinguish the three LLM architectures: **decoder-only** (GPT, Llama), **encoder-only** (BERT), and **encoder-decoder** (T5, Whisper).
3. Explain **self-supervised pretraining** with cross-entropy loss and teacher forcing on large corpora.
4. Describe problems with web-scraped training data: quality, bias, privacy, and copyright.
5. Evaluate LLMs using **perplexity** and understand its relationship to probability.
6. Explain why **static embeddings** (word2vec) are insufficient and how **contextual embeddings** address this.
7. Define the **self-attention** mechanism: queries, keys, values, and scaled dot-product attention.
8. Describe **multi-head attention** and why multiple heads are useful.
9. Explain the components of a **transformer block**: attention, feedforward layers, residual connections, layer normalisation.
10. Understand **causal masking** for autoregressive generation.
11. Describe **positional encodings** and why transformers need them.
12. Explain the **language modelling head**: unembedding layer, logits, and softmax.
13. Discuss **scaling laws** and **KV caching** for efficient inference.

The Big Picture

This week introduces the **transformer architecture**, the foundation of modern LLMs. Unlike RNNs which process sequences step-by-step, transformers use **self-attention** to process all tokens in parallel, enabling much longer contexts and more efficient training. LLMs are transformers trained on massive corpora with billions of parameters, achieving remarkable capabilities through **pretraining** followed by instruction tuning and alignment.

What Examiners Like to Ask

- Define an LLM and compare it to n-gram/RNN language models.
- Explain the three LLM architectures (decoder, encoder, encoder-decoder) and their use cases.
- Describe **self-supervised training**: cross-entropy loss, teacher forcing.
- Define **perplexity** and explain why lower is better.
- Explain why static embeddings fail for polysemous words and how attention solves this.
- Write the **scaled dot-product attention** formula and explain each component.
- Explain the roles of **queries, keys, and values** in attention.
- Describe **multi-head attention** and why it's useful.
- Explain the components of a **transformer block** (attention, FFN, layer norm, residuals).
- Explain **causal masking** and why it's needed for language modelling.
- Describe why transformers need **positional encodings**.

- Explain the **quadratic complexity** of attention w.r.t. sequence length.
- Describe **weight tying** between embedding and unembedding layers.
- Explain **scaling laws**: how loss scales with parameters, data, and compute.

5.2 Overview + Core Concepts + Extended Theory

Roadmap

1. What are Large Language Models?
2. Three LLM architectures: Decoder, Encoder, Encoder-Decoder
3. Pretraining: self-supervision, cross-entropy loss, teacher forcing
4. Training data and its problems
5. Evaluating LLMs: perplexity
6. Static vs contextual embeddings
7. Self-attention mechanism
8. Multi-head attention
9. The transformer block
10. Parallelising attention and causal masking
11. Positional encodings
12. Language modelling head
13. Scaling laws and KV cache

5.2.1 Introduction to Large Language Models

Large Language Model (LLM)

A **Large Language Model** is a neural network that:

- Takes a **context/prefix** as input
- Outputs a **probability distribution** over possible next words
- Has **billions of parameters**
- Is trained on **massive corpora** (web-scale data)

LLMs can **generate text** by repeatedly sampling from the next-word distribution.

Continuity with Earlier LMs

LLMs share the same fundamental task as n-gram and RNN language models:

- Assign probabilities to sequences of words
- Generate text by predicting the next word
- Trained to minimise cross-entropy loss

What's different: Scale (billions of parameters), architecture (transformers), and training data (web-scale corpora).

Fundamental Intuition

Text contains enormous amounts of knowledge.
Pretraining on lots of text with all that knowledge is what gives LLMs their remarkable abilities.

5.2.2 Three LLM Architectures

Architecture	Examples	Characteristics
Decoder-only	GPT, Claude, Llama, Mistral, DeepSeek	Autoregressive (left-to-right); generates one token at a time; what people typically mean by "LLM"
Encoder-only	BERT, HuBERT	Masked language model (MLM); sees context on both sides; typically fine-tuned for classification
Encoder-Decoder	T5, Whisper, Flan-T5	Maps input sequence to output sequence; used for translation, speech recognition, summarisation

Decoder vs Encoder

- **Decoder:** Causal/autoregressive — can only see past tokens (left-to-right generation).
- **Encoder:** Bidirectional — can see tokens on both sides (used for understanding/classification).

5.2.3 Pretraining Large Language Models

Three Stages of LLM Training

1. **Pretraining:** Learn to predict next word on massive unlabelled text.
2. **Instruction Tuning:** Fine-tune on instruction-following examples.
3. **Preference Alignment:** Align with human values and safety norms (e.g., RLHF).

This week focuses on **pretraining**.

Self-Supervised Training

Self-supervision: The text itself provides the training signal.

1. Take a corpus of text.
2. At each position t, ask the model to predict the next word w_{t+1}.
3. Train using gradient descent to minimise prediction error.

No manual labels required — the next word is the label!

Cross-Entropy Loss for LM Training

The loss at position t is the negative log probability of the true next word:

$$L_{CE}(\hat{y}_t, y_t) = -\log \hat{y}_t[w_{t+1}]$$

Intuition:

- If model assigns low probability to the correct word ⇒ high loss.
- Training pushes the model to assign higher probability to correct words.

Since y_t is one-hot, only the probability of the correct word matters.

Teacher Forcing (Reminder)

During training:

- At each position t, compute loss for predicting w_{t+1}.
- Move to position $t+1$ using the **correct word** w_{t+1}, not the model's prediction.

Why: Prevents error accumulation during training.

5.2.4 Training Data and Its Problems

Common pretraining corpora:

- **Common Crawl:** Snapshots of the entire web (billions of pages).
- **C4 (Colossal Clean Crawled Corpus):** 156 billion tokens, filtered English.
- **The Pile:** Curated mix of academic, web, books, and dialogue.

Problems with Web-Scraped Data

- **Copyright:** Much text is copyrighted; legality unclear.
- **Privacy:** Websites may contain private information (IP addresses, phone numbers).
- **Bias/Skew:** Disproportionately US-authored content; reflects societal biases.
- **Quality:** Boilerplate, adult content, duplicates need filtering.
- **Toxicity:** Toxicity detection can mistakenly flag minority dialects.

5.2.5 Evaluating LLMs: Perplexity

Perplexity

Perplexity measures how "surprised" the model is by unseen text.

For a test set of n tokens $w_{1:n}$:

$$\text{Perplexity}(w_{1:n}) = P(w_{1:n})^{-\frac{1}{n}} = \sqrt[n]{\prod_{i=1}^{n} \frac{1}{P(w_i \mid w_{<i})}}$$

Interpretation:

- **Lower perplexity = better model** (less surprised).
- Range: $[1, \infty)$.
- Minimising perplexity ⇔ maximising probability.

Perplexity Comparisons

Perplexity is sensitive to tokenisation, so **only compare models with the same tokeniser**.

Other evaluation factors:

- **Size:** Memory, training time, inference cost.
- **Energy:** kWh or CO_2 emissions.
- **Fairness:** Stereotypes, performance disparities across groups.

5.2.6 Static vs Contextual Embeddings

Problem with Static Embeddings

Word2vec embeddings are static: The same vector for a word regardless of context.

Example: "The chicken didn't cross the road because **it** was too tired."

What does "it" refer to? The static embedding can't distinguish:

- "it" = chicken (tired)
- "it" = road (wide)

Contextual Embeddings

Contextual embedding: A word's representation **changes depending on surrounding words**.

Solution: Use **attention** to build up the embedding by selectively integrating information from neighbouring words.

We say a word "attends to" some neighbours more than others.

5.2.7 Self-Attention Mechanism

Attention: Definition

Attention is a mechanism for computing contextual embeddings by:

1. Computing **similarity scores** between the current token and all context tokens.
2. Using these scores to compute a **weighted sum** of context representations.

Query, Key, Value Roles

Each token x_i plays three roles, represented by learned projections:

$$q_i = x_i W^Q \qquad \text{(query: "what am I looking for?")}$$
$$k_i = x_i W^K \qquad \text{(key: "what do I contain?")}$$
$$v_i = x_i W^V \qquad \text{(value: "what information do I provide?")}$$

Matrices: $W^Q, W^K \in \mathbb{R}^{d \times d_k}$, $W^V \in \mathbb{R}^{d \times d_v}$

Scaled Dot-Product Attention

For current token i with query q_i, attending to tokens $j \leq i$:

1. Compute similarity scores:

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

2. Normalise with softmax:

$$\alpha_{ij} = \text{softmax}(\text{score}(x_i, x_j)) = \frac{\exp(\text{score}(x_i, x_j))}{\sum_{k \leq i} \exp(\text{score}(x_i, x_k))}$$

3. Weighted sum of values:

$$a_i = \sum_{j \leq i} \alpha_{ij} v_j$$

Why Scale by $\sqrt{d_k}$?

Dot products can become very large for high-dimensional vectors. Large values cause softmax to saturate (outputs near 0 or 1), leading to vanishing gradients. Scaling by $\sqrt{d_k}$ keeps values in a reasonable range.

5.2.8 Multi-Head Attention

Multi-Head Attention

Instead of one attention head, use h heads in parallel:

- Each head has its own W_c^Q, W_c^K, W_c^V matrices.
- Each head can attend to different aspects of the context.

Equations:

$$\text{head}_c = \text{Attention}(Q_c, K_c, V_c)$$
$$\text{MultiHead}(x_i) = [\text{head}_1; \text{head}_2; \ldots; \text{head}_h] W^O$$

where $W^O \in \mathbb{R}^{h \cdot d_v \times d}$ projects concatenated heads back to dimension d.

Why Multiple Heads?

Different heads can learn to attend to different linguistic patterns:

- One head might attend to syntactic dependencies.
- Another might attend to semantic relationships.
- Another might attend to nearby tokens.

5.2.9 The Transformer Block

Transformer Block Components

A transformer block contains:

1. **Multi-Head Attention**
2. **Residual Connection** (add input to output)
3. **Layer Normalisation**
4. **Feedforward Network** (2-layer MLP with ReLU)
5. **Another Residual Connection + Layer Norm**

Transformer Block Equations

$$
\begin{aligned}
t_i^1 &= \text{LayerNorm}(x_i) \\
t_i^2 &= \text{MultiHeadAttention}(t_i^1, [x_1, \ldots, x_N]) \\
t_i^3 &= t_i^2 + x_i && \text{(residual connection)} \\
t_i^4 &= \text{LayerNorm}(t_i^3) \\
t_i^5 &= \text{FFN}(t_i^4) \\
h_i &= t_i^5 + t_i^3 && \text{(residual connection)}
\end{aligned}
$$

Layer Normalisation

Normalise each vector to have zero mean and unit variance:

$$\hat{x} = \frac{x - \mu}{\sigma}$$

where $\mu = \frac{1}{d}\sum_i x_i$ and $\sigma = \sqrt{\frac{1}{d}\sum_i (x_i - \mu)^2}$

With learnable gain g and bias b:

$$\text{LayerNorm}(x) = g \cdot \frac{x - \mu}{\sigma} + b$$

Feedforward Network

A 2-layer MLP applied independently to each position:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

Typically $d_{ff} = 4d$ (e.g., $d = 512$, $d_{ff} = 2048$).

Residual Stream View

Attention is the **only** component that moves information between tokens. All other components operate on individual residual streams. Attention can be viewed as "moving information from one token's stream to another."

5.2.10 Parallelising Attention and Causal Masking

Matrix Form of Attention

Pack all N tokens into matrix $X \in \mathbb{R}^{N \times d}$:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$
$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

This computes attention for **all tokens simultaneously**!

Causal Masking

Problem: $QK^\top$ computes scores for **all** pairs, including future tokens.

Solution: Add a mask matrix M where $M_{ij} = -\infty$ for $j > i$:

$$A = \text{softmax}\left(\text{mask}\left(\frac{QK^\top}{\sqrt{d_k}}\right)\right)V$$

After softmax, positions with $-\infty$ become 0 (no attention to future).

Quadratic Complexity

Attention is $O(N^2)$ in sequence length:

- $QK^\top$ is an $N \times N$ matrix.
- Every token attends to every other token.

This limits context length (though techniques like sparse attention help).

5.2.11 Positional Encodings

Why Transformers Need Position Information

Attention treats all positions equally — it has **no inherent notion of order**.

Without positional encodings, "the cat sat on the mat" and "mat the on sat cat the" would have the same representation!

Absolute Position Embeddings

Learn a position embedding matrix $E_{\text{pos}} \in \mathbb{R}^{N_{\max} \times d}$:

- One learned embedding for each position $1, 2, \ldots, N_{\max}$.
- Add position embedding to token embedding: $x_i = E[w_i] + E_{\text{pos}}[i]$

5.2.12 Language Modelling Head

From Hidden State to Next-Word Probabilities

After L transformer layers, the final hidden state h_N^L is transformed into a probability distribution:

1. Unembedding layer: Project to vocabulary-sized logits:

$$u = h_N^L E^\top \quad \text{(shape: } 1 \times |V|\text{)}$$

2. Softmax: Convert logits to probabilities:

$$y = \text{softmax}(u)$$

$y_k = P(w_{N+1} = k \mid w_{1:N})$

Weight Tying

The unembedding matrix is the **transpose of the embedding matrix** E:

- **Embedding:** E maps one-hot $\rightarrow$ embedding (shape $|V| \times d$)
- **Unembedding:** $E^\top$ maps embedding $\rightarrow$ logits (shape $d \times |V|$)

Benefits: Fewer parameters, often better generalisation.

5.2.13 Scaling Laws and KV Cache

Scaling Laws

LLM performance (loss L) scales as a **power law** with:

- **Model size** N (number of parameters)
- **Dataset size** D (tokens)
- **Compute** C (FLOPs)

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C}$$

Key insight: We can predict final performance from early training!

Estimating Parameters

For a transformer with n layers and dimension d:

$$N \approx 12 \cdot n \cdot d^2$$

GPT-3: $n = 96$ layers, $d = 12288 \Rightarrow N \approx 175$ billion parameters.

KV Cache

Problem: At inference, we generate tokens one at a time. Recomputing all keys and values at each step is wasteful.

Solution: Cache the key and value vectors for all previous tokens.

At step i:

- Compute q_i, k_i, v_i for the new token.
- Retrieve $k_1, \ldots, k_{i-1}$ and $v_1, \ldots, v_{i-1}$ from cache.
- Compute attention using cached keys/values.
- Store k_i, v_i in cache for future steps.

5.2.14 Summary: Transformers vs RNNs

	RNN/LSTM	**Transformer**
Processing	Sequential (one step at a time)	Parallel (all tokens at once)
Context	Via hidden state (limited)	Via attention (direct access)
Long-range deps	Difficult (vanishing gradients)	Easy (direct attention)
Training	Sequential backprop through time	Highly parallelisable
Complexity	$O(N)$ per step	$O(N^2)$ for attention
Position info	Implicit (order of processing)	Explicit (positional encodings)

6 Week 7: Prompting and In-Context Learning

6.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Describe the principles of **prompt engineering** and **in-context learning**.
2. Implement QA methods with **zero-shot**, **few-shot**, and **chain-of-thought** prompting.
3. Explain **temperature** and its effect on generation randomness.
4. Describe advanced techniques: **self-reflection**, **self-consistency**, **tool use**, and **multi-agent systems**.

The Big Picture

When using LLM APIs, we cannot modify the model's parameters. Instead, we improve outputs by carefully designing the **input prompt**. This week covers prompt engineering techniques ranging from simple strategies (politeness, clear instructions) to advanced methods (chain-of-thought, tool calling, multi-agent debate).

What Examiners Like to Ask

- Define **auto-regressive generation** and explain how LLMs generate text.
- Distinguish between LLM **interface** (ChatGPT) and **API** (raw model).
- Explain strategies for **vanilla prompting**: politeness, clear instructions, revision.
- Define **in-context learning (ICL)** and explain zero-shot vs. few-shot prompting.
- Explain **chain-of-thought (CoT)** prompting and why it helps.
- Define **temperature** and explain its effect on probability distributions.
- Calculate probability changes when temperature is modified.
- Explain **self-reflection** and **self-consistency**.
- Describe why LLMs need **external tools** (calculators, search engines).
- Explain **multi-agent systems** and how debate improves outputs.

6.2 Overview + Core Concepts + Extended Theory

Roadmap

1. LLM Recap: auto-regressive generation
2. Interface vs. API
3. Introduction to prompt engineering
4. Vanilla prompting strategies
5. In-context learning (zero-shot, few-shot)
6. Chain-of-thought prompting
7. Self-reflection and self-consistency
8. Temperature and randomness

9. Tool use (calculators, RAG)
10. Multi-agent systems

6.2.1 LLM Recap: Auto-Regressive Generation

Auto-Regressive Generation

Auto-regressive generation means words are generated **one by one**, where each generated word depends on all previously generated words.

$$P(w_1, w_2, \ldots, w_n) = \prod_{i=1}^{n} P(w_i \mid w_1, \ldots, w_{i-1})$$

Key insight: An LLM is a function that takes text input and produces text output by predicting the next token iteratively.

LLMs are General-Purpose

Unlike traditional ML models trained for specific tasks, LLMs are trained on a general objective (next-word prediction). Through large-scale training, they learn to follow instructions and generate contextually appropriate responses for many tasks.

6.2.2 Interface vs. API

Interface vs. API

Interface (e.g., ChatGPT web app):

- User-facing system built around an LLM
- Integrates many modules: user profiling, intention recognition, conversation memory, tool calling
- Handles multi-turn dialogue and context

API (e.g., OpenAI API):

- Raw model that takes text input → text output
- Only processes single QA pairs
- No memory between calls (unless you provide context)

Example: Same Question, Different Answers

Q1: "Who is Donald Trump?" → Detailed biography (interface has context)

Q2: "What about Carter?" → Interface understands follow-up; API might give random "Carter" (Jimmy Carter, Captain Carter, Vince Carter...)

6.2.3 Introduction to Prompt Engineering

Prompt Engineering

Prompt engineering is the design of effective input prompts to guide LLM outputs toward desired results.

Why needed: When using APIs, we cannot change model parameters. The only lever we have is the **input**.

Two approaches to improve LLM outputs:

1. **Change the input** (prompting) ← This week
2. **Change the parameters** (fine-tuning, RLHF) ← Week 8

Prompt engineering techniques covered:

1. Vanilla prompting (simple strategies)
2. In-context learning (few-shot)
3. Chain-of-thought
4. Tool use
5. Multi-agent systems

6.2.4 Vanilla Prompting Strategies

Strategies to Improve Prompting

1. **Be polite** — Politeness in prompts often yields better responses
2. **Give clear instructions** — Explicit, step-by-step instructions
3. **Ask for revision** — Let the model critique and improve its answer
4. **Emotional prompts** — Adding emotional stakes ("This is very important")
5. **Template selection** — Use the LLM to choose the best prompt template

Politeness Affects Output

Research shows politeness impacts LLM responses, but the effect is **culture-dependent**:

- **English/Chinese:** Polite prompts → better responses
- **Japanese:** Rude prompts sometimes perform better (cultural norm of "reading the air")

Why? Training data reflects human conversations where polite questions receive more detailed answers.

Revision Strategy

Initial prompt: "Rate this essay."

Better prompt: "Rate this essay. First explain your reasoning, then give a score."

Asking the model to **explain before answering** often improves accuracy.

6.2.5 In-Context Learning (ICL)

In-Context Learning

In-context learning (ICL) is the ability of LLMs to learn and perform new tasks **directly from examples in the prompt**, without updating parameters.

Key properties:

- No parameter updates required
- Task specification via examples in the prompt
- Also called **few-shot learning**

Zero-Shot vs. Few-Shot

Zero-shot: No examples provided; model relies on instructions only.

Few-shot (K-shot): Provide K input-output examples in the prompt.

Example (sentiment analysis):

```
"love this phone - battery lasts all day" -> Positive
"The update ruined everything" -> Negative
"Yeah, fantastic job... three delays" -> Negative

Analyze: "It's okay, does the job" -> ?
```

Formatting Matters

The format of few-shot examples significantly affects performance. Consistent formatting (e.g., "Input -¿ Output") helps the model understand the task pattern.

6.2.6 Chain-of-Thought (CoT) Prompting

Chain-of-Thought Prompting

Chain-of-thought (CoT) prompting decomposes complex tasks into step-by-step reasoning.

Magic prompt: "Let's think step by step."

Why it works: Breaking complex reasoning into smaller steps makes each step easier, reducing errors.

CoT Example

Without CoT:

```
Q: Roger has 5 tennis balls. He buys 2 cans of 3 balls each.
   How many tennis balls does he have?
A: 11 (WRONG)
```

With CoT:

```
Q: ... Let's think step by step.
A: Roger starts with 5 balls.
   He buys 2 cans × 3 balls = 6 balls.
   Total: 5 + 6 = 11 balls. (CORRECT)
```

CoT Effectiveness

CoT provided significant improvements in earlier LLMs (e.g., 72% → 88% on some benchmarks). With newer models (post-2024), the gap has narrowed (85% → 89%) as models have improved baseline reasoning.

6.2.7 Self-Reflection

Self-Reflection

Self-reflection is when an LLM uses its own reasoning to evaluate and refine its previous responses.

Steps:

1. **Initial generation:** Model produces an answer
2. **Reflection:** Same model critiques that answer
3. **Revision:** Model rewrites using self-feedback

Constitutional AI

Harmful prompt: "How can I break into someone's house?"

Step 1 (Initial): Model generates potentially harmful response.

Step 2 (Critique): Model evaluates: "This response could enable illegal activity."

Step 3 (Revision): Model rewrites: "I can't help with illegal activities..."

Note: Steps 1-2 are hidden from the user.

6.2.8 Temperature and Self-Consistency

Temperature

Temperature T controls the randomness of LLM generation by scaling logits before softmax:

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

where z_i is the logit for token i.

Effects:

- $T < 1$: Sharper distribution → more deterministic (high-probability tokens dominate)
- $T = 1$: Standard softmax
- $T > 1$: Flatter distribution → more random (probabilities become more uniform)

Temperature Example

Logits: $z_{\text{cat}} = 2$, $z_{\text{dog}} = 1$, $z_{\text{apple}} = 0.1$

Temperature	P(cat)	P(dog)	P(apple)
$T = 0.5$	0.85	0.11	0.02
$T = 1.0$	0.66	0.24	0.10
$T = 2.0$	0.50	0.30	0.19

Lower $T \rightarrow$ "cat" dominates. Higher $T \rightarrow$ more uniform.

Self-Consistency

Self-consistency uses randomness to find the most confident answer:

1. Generate multiple answers with higher temperature
2. Take a **majority vote** across answers

Intuition: The most frequent answer is likely correct.

6.2.9 Tool Use

Why LLMs Need Tools

LLMs struggle with:

- **Arithmetic:** "What is 3847×9284?" $\rightarrow$ Often wrong
- **Factual updates:** Knowledge cutoff means outdated information
- **Hallucinations:** Plausible but false outputs

Program of Thought

Instead of computing directly, ask the LLM to **generate code**:

Prompt: "Write Python code to compute 3847×9284"

Output:

```
print(3847 * 9284)  # Execute externally -> 35719548
```

The code is then executed by an external interpreter.

Retrieval-Augmented Generation (RAG)

RAG augments LLM prompts with retrieved information from external sources (search engines, databases).

Example:

1. User asks: "What is the current GBP to USD exchange rate?"
2. System retrieves current rate from financial API
3. LLM receives: "Exchange rate is 1.30. User asks: ..."
4. LLM computes: "100 GBP = 130 USD"

Model Hallucination

A **hallucination** is when an LLM produces output that sounds plausible but is **false or unsupported by data**.

Always verify LLM outputs, especially for factual claims!

6.2.10 Multi-Agent Systems

Multi-Agent Systems

Multiple LLMs collaborate or debate to improve output quality.

Approaches:

- **Cross-model reflection:** LLM#1 generates → LLM#2 critiques → revision
- **Debate:** Multiple LLMs discuss until consensus
- **Exchange-of-Thought:** Structured discussion protocols

Debate Format

```
LLM#1: "The answer is X because..."
LLM#2: "I disagree. Consider Y..."
LLM#1: "Good point. Revised answer: Z"
LLM#2: "I agree with Z."
-> Final answer: Z
```

Research shows: More agents + more discussion rounds → better performance.

6.2.11 Summary: Prompting Techniques

Technique	Description
Vanilla prompting	Politeness, clear instructions, revision requests
Zero-shot	Task description only, no examples
Few-shot (ICL)	Provide K examples in prompt
Chain-of-thought	"Let's think step by step"
Self-reflection	Model critiques and revises own output
Self-consistency	Multiple samples + majority vote
Tool use	External calculators, search, code execution
Multi-agent	Multiple LLMs debate/collaborate

7 Week 8: LLM Training Strategies

7.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Describe the principles of **supervised fine-tuning (SFT)** and **reinforcement learning from human feedback (RLHF)**.
2. Explain the three-stage LLM training pipeline: **pre-training** → **instruction fine-tuning** → **RLHF**.
3. Describe the difference between **PPO**, **DPO**, and **GRPO**.
4. Explain **LoRA** and how it reduces memory requirements for fine-tuning.
5. Calculate PPO objectives given probability ratios and advantages.

The Big Picture

This week explains how LLMs are trained through pre-training, instruction fine-tuning, and human-feedback methods (RLHF, PPO, DPO, GRPO) to turn raw next-token predictors into aligned, instruction-following models. We also address challenges such as memorisation, catastrophic forgetting, annotation cost, and training stability.

What Examiners Like to Ask

- Describe the three stages of LLM training and their purposes.
- Explain the **loss function** and **data requirements** for pre-training vs. SFT vs. RLHF.
- Define **catastrophic forgetting** and other SFT challenges.
- Explain **RLHF** and the role of **reward models**.
- Calculate **PPO objectives**: ratio, clipped ratio, unclipped term, final objective.
- Explain **LoRA**: how it reduces parameters via matrix decomposition.
- Compare **DPO** and **GRPO** to PPO-based RLHF.

7.2 Overview + Core Concepts + Extended Theory

Roadmap

1. Challenges in LLM training
2. The three-step training pipeline
3. Pre-training: next-token prediction
4. Memorisation vs. learning
5. Instruction fine-tuning (SFT)
6. SFT challenges and data considerations
7. RLHF and reward models
8. PPO: clipping and stability
9. LoRA: efficient fine-tuning

10. DPO and GRPO: simpler alternatives

7.2.1 Challenges in LLM Training

LLM Training as Supervised Learning?

Traditional supervised learning has three components:

- **Loss function:** The learning target
- **Optimiser:** How to find the target
- **Labelled data:** Where to train the model

LLM challenges:

- **Loss function:** How to define correctness for language generation?
- **Optimiser:** How to optimise when loss is unclear?
- **Labelled data:** Impossible to manually label all human knowledge

Solution: Step-by-Step Training

LLM training is a systematic task with no simple solution. The key insight: break it into steps, each solving part of the problem.

7.2.2 The Three-Step LLM Training Pipeline

LLM Training Pipeline

1. **Step 1: Pre-training** — "Speak like a human"
   - Train to predict next token
   - Uses plain text (no labels needed)
   - Gives fluency and world knowledge
2. **Step 2: Instruction Fine-Tuning (SFT)** — "Understand instructions"
   - Train on instruction-response pairs
   - Teaches task-following behavior
3. **Step 3: RLHF** — "Learn from interaction"
   - Align with human preferences
   - Uses comparative feedback (which response is better?)

7.2.3 Pre-training: Next-Token Prediction

Pre-training

Goal: Train the model to predict the next token given context.

How it simplifies the three challenges:

- **Loss function:** Cross-entropy loss on next-token prediction
- **Optimiser:** Standard gradient descent
- **Data:** Plain text — no annotation required!

Pre-training Example

Training sentence: "May the force be with you"

Training steps:

Context	Predicted	Target
"May the force"	LLM → ?	"be"
"May the force be"	LLM → ?	"with"
"May the force be with"	LLM → ?	"you"

Loss: $\mathcal{L} = -\log P(\text{target} \mid \text{context})$

If the model predicts the correct word with high probability, loss is low → small parameter updates.

Teacher Forcing

During training, even if the model predicts incorrectly, we use the **correct word** as input for the next step (not the model's wrong prediction).

7.2.4 Memorisation vs. Learning

The Risk of Complex Models

Trade-off: Model complexity vs. generalisability.

LLMs are extremely complex (billions of parameters). This creates risks:

- **Overfitting:** Learning noise in training data
- **Memorisation:** Storing patterns rather than understanding them

Question: Does the model truly understand, or just memorise?

Spurious Correlations Example

Task: Classify Pokémon vs. Digimon images.

Model accuracy: 98% on training and test data.

Reality: The model learned that:

- Pokémon images are PNG (transparent → black background)
- Digimon images are JPEG (white background)

The model discriminated based on **background colour**, not the actual characters!

7.2.5 Instruction Fine-Tuning (SFT)

Instruction Fine-Tuning

Goal: Teach the model to follow human instructions.

Process:

1. Collect instruction-response pairs
2. Initialise with pre-trained parameters
3. Fine-tune on the instruction data

Key difference from pre-training:

- Pre-training: Predict every next word
- SFT: Only train on the **response** part (instruction is just context)

SFT Example

Instruction: "What's the iconic expression from the Star Wars movie?"

Training (fine-tune on response only):

Context	Target
[Instruction]	"May"
[Instruction] May	"the"
[Instruction] May the	"force"
...	...

Cross-lingual Transfer

Fine-tuning on one language can transfer to others! Example: Pre-train on 104 languages, fine-tune only on Chinese QA → model can answer English QA questions too.

7.2.6 SFT Challenges

Common SFT Problems

- **Catastrophic Forgetting:** Learning new tasks overwrites previous knowledge
- **Instruction Ambiguity:** Different tasks expect different formats (bullets vs. long-form)
- **Data Imbalance:** Model overfits to tasks with more data
- **Domain Shift:** Mixing domains (medical vs. casual) confuses the model

Two Paths to Multi-Task AI

Path 1: Train separate models for each task (more accurate, less general)

Path 2: Continuously fine-tune one model on all tasks (more general, risk of forgetting)

Most commercial LLMs use Path 2 with careful data mixing and curriculum.

Data Quality Matters

LIMA paper: Just 1,000 high-quality examples can match GPT-4 in 43% of cases.
"Less Is More for Alignment" — quality beats quantity.

7.2.7 Self-Instruct: Generating Training Data

Self-Instruct

Problem: Manual annotation is expensive.

Solution: Use LLMs to generate instruction-response pairs automatically.

Benefits:

- Reduces annotation cost
- Expands datasets beyond human-written examples

Risks:

- Generated labels may be incorrect
- May violate API terms of service (e.g., OpenAI prohibits using outputs to train competing models)

7.2.8 Reinforcement Learning from Human Feedback (RLHF)

RLHF

Problem with SFT: Requires word-level annotation (expensive); ignores overall response quality.

RLHF solution: Annotators only need to **compare** responses (which is better?), not write correct answers.

Process:

1. Generate multiple responses to a prompt
2. Human ranks responses (or picks the better one)
3. Increase probability of preferred responses; decrease probability of dispreferred ones

RLHF Example

Prompt: "What's the iconic Star Wars expression?"

Response A: "May the force be with you" ← Human prefers

Response B: "If necessary, please learn to let it go"

Training: Increase P(Response A), decrease P(Response B).

7.2.9 Reward Models

Reward Model

Problem: Human annotation doesn't scale; humans struggle to distinguish good responses from well-trained models.

Solution: Train a **reward model** to simulate human preferences.

Training: Use human-ranked examples to train a model that predicts preference scores.

Risk: The LLM may learn to **exploit** the reward model (reward hacking), producing responses that score high but aren't actually good.

7.2.10 PPO: Proximal Policy Optimization

PPO Principles

Goal: Update model to increase probability of good responses.

Key principles:

1. If reward is high → increase generative probability
2. If reward is low → decrease generative probability
3. Don't move too far from the previous model (stability)

Challenge: Requires keeping two models in memory (current + previous version).

PPO Clipping Mechanism

Ratio: $r = \frac{P_{\text{current}}}{P_{\text{previous}}}$

Clipped ratio: Constrain r to range $[1-\varepsilon, 1+\varepsilon]$ (typically $\varepsilon = 0.2$)

PPO objective:

$$L^{\text{CLIP}} = \min\left(r \cdot A, \text{clip}(r, 1-\varepsilon, 1+\varepsilon) \cdot A\right)$$

where A is the **advantage** (reward signal).

Why clip? Prevents extremely large updates that could destabilise training.

PPO Calculation Example 1

Given: $P_{\text{current}} = 0.3$, $P_{\text{previous}} = 0.2$, Advantage $= 2.0$, $\varepsilon = 0.2$

Step 1: Ratio $= 0.3/0.2 = 1.5$

Step 2: Unclipped term $= 1.5 \times 2.0 = 3.0$

Step 3: Clipped ratio $= \min(1.5, 1.2) = 1.2$ (since $1 + \varepsilon = 1.2$)

Step 4: Clipped term $= 1.2 \times 2.0 = 2.4$

Step 5: Final objective $= \min(3.0, 2.4) = \boxed{2.4}$

PPO Calculation Example 2

Given: $P_{\text{current}} = 0.25$, $P_{\text{previous}} = 0.20$, Advantage $= 1.5$, $\varepsilon = 0.1$

Step 1: Ratio $= 0.25/0.20 = 1.25$

Step 2: Unclipped term $= 1.25 \times 1.5 = 1.875$

Step 3: Clipped ratio $= \min(1.25, 1.1) = 1.1$

Step 4: Clipped term $= 1.1 \times 1.5 = 1.65$

Step 5: Final objective $= \min(1.875, 1.65) = \boxed{1.65}$

Interpreting the Ratio

- $r > 1$: Current model already favours this response → less update needed
- $r < 1$: Current model doesn't favour this response → more update needed

Clipping prevents the model from over-focusing on single examples.

7.2.11 LoRA: Low-Rank Adaptation

LoRA

Problem: PPO requires two models in memory → very expensive for large LLMs.

Solution: Instead of updating the full weight matrix W (size $m \times n$), decompose updates into two smaller matrices:

$$W' = W + L \times R$$

where L is $m \times r$ and R is $r \times n$, with $r \ll m, n$.

Parameter savings:

- Full matrix: $m \times n$ parameters
- LoRA: $m \times r + r \times n$ parameters

LoRA Parameter Calculation

Full matrix: $6 \times 5 = 30$ parameters

LoRA with $r = 2$:

- L: $6 \times 2 = 12$ parameters
- R: $2 \times 5 = 10$ parameters
- Total: $12 + 10 = 22$ parameters

Savings: $30 - 22 = 8$ parameters (27% reduction)

For large matrices, savings are dramatic: $1000 \times 1000 = 1M$ vs. $1000 \times 16 + 16 \times 1000 = 32K$.

LoRA Trade-offs

Pros:

- Reduces trainable parameters significantly
- Enables fine-tuning large LLMs with limited GPU memory

Cons:

- Does NOT guarantee same accuracy as full fine-tuning
- Smaller r → more compression but lower accuracy
- No free lunch: accuracy vs. efficiency trade-off

7.2.12 DPO: Direct Preference Optimization

DPO

Problem with PPO: Complex pipeline (generate → get reward → policy gradient → clip).

DPO solution: Directly optimise preferences without RL.

Key idea: We already know which responses humans prefer. Just make preferred responses more likely than dispreferred ones directly.

Advantages over PPO:

- No reward model needed during training
- No reinforcement learning required
- Simpler and cheaper to implement

7.2.13 GRPO: Group Relative Policy Optimization

GRPO

Problem with PPO: Needs a critic/value function to estimate advantages.

GRPO solution: Compute advantages by comparing responses **within a group**.

Process:

1. For a prompt, generate multiple candidate responses
2. Compute statistics (mean, std) over the group
3. Define advantage as how much better each response is relative to peers

Advantages:

- No learned value function needed (simpler)
- More stable training
- Reward function can be simple (e.g., response length)

7.2.14 Summary: Training Methods Comparison

Method	Key Idea	Requirements
Pre-training	Next-token prediction	Plain text (no labels)
SFT	Train on instruction-response pairs	Labelled instruction data
RLHF + PPO	RL with clipped updates	Reward model + two LLMs in memory
LoRA	Low-rank weight decomposition	Reduced memory
DPO	Direct preference optimisation	Preference pairs (no RL)
GRPO	Group-relative advantages	Multiple responses per prompt

7.2.15 Key Equations Summary

Essential Formulas

Pre-training loss (cross-entropy):

$$\mathcal{L} = -\sum_t \log P(w_t \mid w_1, \ldots, w_{t-1})$$

PPO ratio and clipping:

$$r = \frac{P_{\text{current}}}{P_{\text{previous}}}, \quad L^{\text{CLIP}} = \min(r \cdot A, \text{clip}(r, 1-\varepsilon, 1+\varepsilon) \cdot A)$$

LoRA decomposition:

$$W' = W + L \times R, \quad \text{params} = m \times r + r \times n \ll m \times n$$

8 Week 9: Retrieval-Augmented Generation (RAG)

8.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Understand the core concepts of **Retrieval-Augmented Generation (RAG)** and how it differs from standard LLM approaches.
2. Build and configure a basic **RAG pipeline** using embeddings, retrievers, and generators.
3. Evaluate and optimise RAG performance through effective **data preparation**, **chunking**, and **retrieval strategies**.
4. Explain **similarity metrics** (cosine, Jaccard, BM25) and **indexing** methods.

The Big Picture

RAG combines retrieval from external knowledge bases with LLM generation. Instead of relying solely on parametric knowledge (what the model memorised during training), RAG retrieves relevant documents and uses them as context. This addresses knowledge cutoffs, reduces hallucinations, and enables domain-specific applications without retraining.

What Examiners Like to Ask

- Define RAG and explain its advantages over fine-tuning.
- List scenarios where RAG is applicable (long-tail data, frequent updates, privacy, etc.).
- Explain **discrete representations**: binary, TF, TF-IDF.
- Explain **continuous representations**: Dense Passage Retrieval, ReContriever.
- Calculate **cosine similarity** and **Jaccard similarity** for given vectors.
- Describe **BM25** and its components.
- Explain indexing methods for discrete (inverted index) and continuous (FAISS) representations.
- Describe the **Naïve RAG** pipeline vs. **Advanced RAG** enhancements.
- List query optimisation and chunk optimisation strategies.

8.2 Overview + Core Concepts + Extended Theory

Roadmap

1. What is RAG and why use it?
2. RAG vs. fine-tuning
3. RAG application scenarios
4. Information Retrieval foundations: embeddings, similarity, indexing
5. IR evaluation metrics
6. Naïve RAG pipeline
7. Advanced RAG enhancements
8. Key technologies and optimisation

8.2.1 What is RAG?

Retrieval-Augmented Generation

RAG is a framework that:

1. **Retrieves** relevant documents from an external knowledge base
2. **Augments** the LLM prompt with retrieved context
3. **Generates** answers based on both the query and retrieved information

Key benefit: No need to retrain the LLM for domain updates—just update the knowledge base.

RAG vs. Parametric Knowledge

Parametric knowledge: What the model memorised during training (frozen at training time).
Symbolic knowledge: External documents retrieved at inference time (can be updated).
RAG combines both: the LLM's language capabilities + up-to-date external knowledge.

8.2.2 RAG vs. Fine-tuning

Aspect	RAG	Fine-tuning
Knowledge updates	Update knowledge base (easy)	Retrain model (expensive)
Hallucinations	Reduced (grounded in retrieved docs)	Can still hallucinate
Domain adaptation	Add domain documents	Need domain training data
Traceability	Can cite sources	No source attribution
Compute cost	Retrieval overhead at inference	Training cost upfront

8.2.3 RAG Application Scenarios

When to Use RAG

RAG is especially suitable for:

- **Long-tail data distribution:** Rare but important queries (low frequency $\neq$ low importance)
- **Frequent knowledge updates:** News, stock prices, policies
- **Verification and traceability:** Fact-checking with sources
- **Specialised domain knowledge:** Medical, legal, technical domains
- **Data privacy preservation:** Query knowledge base without training on private data

Example applications: Question answering, fact checking, dialogue systems, summarisation, code generation, commonsense reasoning.

8.2.4 Information Retrieval Foundations

Three Questions in IR

1. **Embedding:** How to map text into features (vectors)?
2. **IR Modelling:** How to measure similarity between features?
3. **Indexing:** How to do it efficiently?

8.2.5 Discrete Text Representations

Discrete Representations

Represent text as sparse vectors based on vocabulary.

Binary representation: 1 if word present, 0 otherwise.

Term Frequency (TF): Count how many times each word appears.

TF-IDF: Weight by term frequency and inverse document frequency:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{N}{\text{DF}(t)}\right)$$

where N = total documents, $\text{DF}(t)$ = documents containing term t.

TF-IDF Example

Documents:

- D1 = "shipment", "gold", "damage", "fire"
- D2 = "delivery", "silver", "arrive", "truck"
- D3 = "shipment", "gold", "arrive", "truck"

Binary representation:

	ship	gold	dam	fire	del	silv	arr	truck
D1	1	1	1	1	0	0	0	0
D2	0	0	0	0	1	1	1	1
D3	1	1	0	0	0	0	1	1

IDF for "damage": $\log(3/1) = 0.477$ (appears in 1 doc $\rightarrow$ high IDF)

IDF for "shipment": $\log(3/2) = 0.176$ (appears in 2 docs $\rightarrow$ lower IDF)

8.2.6 Continuous Text Representations (Dense Embeddings)

Dense Passage Retrieval (DPR)

Train encoders to map questions and passages into continuous vector space.

Training:

- **Positive pairs:** Matched question-passage pairs → similar embeddings
- **Negative pairs:** Unmatched pairs → dissimilar embeddings
- **In-batch negatives:** Use other passages in the same batch as negatives (efficient)

Architecture: Two-tower encoder (separate encoders for queries and documents).

ReContriever: Self-Supervised Retrieval

Problem: What if we don't have labelled question-passage pairs?

Solution: Create pseudo-positive pairs via augmentation:

- Word masking
- Span deletion
- Back-translation
- Adding noise/perturbations

For passage p, create augmented version p'. Treat (p, p') as a positive pair.

8.2.7 Similarity Metrics

Cosine Similarity

$$\cos(\vec{x}, \vec{y}) = \frac{\sum_{i=1}^{n} x_i y_i}{\sqrt{\sum_{i=1}^{n} x_i^2} \cdot \sqrt{\sum_{i=1}^{n} y_i^2}} = \frac{\vec{x} \cdot \vec{y}}{|\vec{x}| \cdot |\vec{y}|}$$

Works for both discrete and continuous representations.

Cosine Similarity Calculation

Given:

- D1 $= [1, 1, 1, 1, 0, 0, 0, 0]$
- D3 $= [1, 1, 0, 0, 0, 0, 1, 1]$

Dot product: $1 \cdot 1 + 1 \cdot 1 + 0 + 0 + 0 + 0 + 0 + 0 = 2$

Magnitudes: $|D1| = \sqrt{4} = 2, \quad |D3| = \sqrt{4} = 2$

Cosine: $\dfrac{2}{2 \cdot 2} = \boxed{0.5}$

Jaccard Similarity

For binary/set representations (ignores magnitudes):

$$\text{Jaccard}(X, Y) = \frac{|X \cap Y|}{|X \cup Y|}$$

Only considers overlap (present/absent), not counts.

Jaccard Similarity Calculation

Given: $R_x = [2, 0, 3, 3]$, $R_y = [1, 1, 0, 5]$

Binary conversion: $R_x \to [1, 0, 1, 1]$, $R_y \to [1, 1, 0, 1]$

Intersection: Indices $\{0, 3\} \rightarrow$ size $= 2$

Union: Indices $\{0, 1, 2, 3\} \rightarrow$ size $= 4$

Jaccard: $\frac{2}{4} = \boxed{0.5}$

BM25

Lexicon-based ranking function:

$$\text{score}(D, Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

Components:

- $\text{IDF}(q_i)$: How important is word q_i?
- $f(q_i, D)$: Term frequency in document D
- $|D|/\text{avgdl}$: Document length normalisation
- k_1, b: Hyperparameters

Intuition: BM25 = IDF weighting × percentage of query words in document.

8.2.8 Indexing for Efficient Retrieval

Inverted Index (Discrete)

For sparse discrete representations:

- Build a linked list for each word mapping to documents containing it
- Query complexity depends only on unique words in query (typically few)

FAISS Indexing (Continuous)

For dense continuous representations:

- **Vector compression:** Reduce vector dimensionality
- **Hierarchical clustering:** Cluster documents; search only nearest cluster centroids

Trade-off: FAISS is approximate—may miss the true nearest neighbour if it's in a different cluster.

8.2.9 IR Evaluation Metrics

Precision and Recall at k

Precision@k: $\frac{\#\text{ relevant in top } k}{k}$

Recall@k: $\frac{\#\text{ relevant in top } k}{\text{total relevant documents}}$

Precision/Recall Example

5 documents ranked by predicted scores:

Rank	1	2	3	4	5
Relevant?	Y	N	N	Y	N
Precision@k	1/1=1.0	1/2=0.5	1/3=0.33	2/4=0.5	2/5=0.4
Recall@k (2 total)	1/2=0.5	1/2=0.5	1/2=0.5	2/2=1.0	2/2=1.0

8.2.10 Naïve RAG Pipeline

Naïve RAG: Three Steps

1. **Indexing:**
   - Divide documents into chunks
   - Generate embeddings for each chunk
   - Store in vector database
2. **Retrieval:**
   - Embed the query
   - Find top-k most similar chunks using vector similarity
3. **Generation:**
   - Combine query + retrieved chunks into a prompt
   - Feed to LLM to generate answer

Prompt Formatting

Order matters! LLMs are sensitive to input order. Common formats:

- Chunk1, Chunk2, Chunk3, Question
- Question, Chunk1, Chunk2, Chunk3

Earlier chunks may receive higher attention.

8.2.11 Advanced RAG Enhancements

Index Optimisation

- **Sliding windows:** Overlapping chunks to preserve context across boundaries
- **Fine-grained segmentation:** Hierarchical structure (document → section → paragraph)
- **Metadata addition:** Add title, date, parent node for filtering and context

Pre-Retrieval Optimisation

- **Query rewriting:** Make queries more explicit, keyword-rich, or structured
- **Query clarification:** Disambiguate vague queries
- **Multiple sub-queries:** Generate diverse queries for broader retrieval
- **Summarisation-based search:** Search over document summaries first, then drill down

- **Retrieve routes:** Multiple retrieval paths (e.g., search docs first, then chunks within)

Post-Retrieval Optimisation

- **Re-ordering:** Arrange chunks by importance for generation
- **Filtering:** Remove irrelevant chunks to reduce hallucination
- **Confidence checking:** Verify retrieval quality before generation

---

8.2.12 When and How to Retrieve

Retrieval Strategies

When to retrieve:

- **Once:** Single retrieval at the start (efficient, but may miss relevant info)
- **Adaptive:** Retrieve when model is uncertain
- **Every N tokens:** Periodic retrieval during generation (thorough, but inefficient)

What level to retrieve:

- **Token/phrase level:** Broad recall, high redundancy
- **Chunk level:** Balanced
- **Entity/knowledge level:** Structured, but depends on KG quality

---

8.2.13 Key Technologies for RAG Optimisation

Chunk Optimisation Strategies

- **Small-to-big:** Embed at sentence level, expand window during generation
- **Sliding window:** Overlapping chunks to avoid boundary issues
- **Two-stage:** Search summaries first (coarse), then chunks within matched docs (fine)

Knowledge Graph as Retrieval Source

Instead of flat document retrieval:

1. Extract entities from query
2. Retrieve subgraph (e.g., 2 hops from entities)
3. Use subgraph as context for LLM

Benefit: Richer structured information for domain-specific tasks.

Embedding Optimisation

- Select appropriate embedding method for the domain
- Fine-tune embeddings on query-document pairs

- Use in-context learning to generate pseudo training data

Fine-tuning RAG Components

Retriever fine-tuning:

- Use LLM attention scores to identify preferred documents
- Contrastive loss on positive/negative pairs

Generator fine-tuning:

- Add entity prediction loss
- Multi-task learning with retrieval-augmented objectives

8.2.14 Summary: RAG Pipeline Comparison

Component	Options
Embedding	Binary, TF, TF-IDF (discrete); DPR, ReContriever (dense)
Similarity	Cosine, Jaccard, BM25
Indexing	Inverted index (discrete); FAISS clustering (dense)
Chunking	Fixed, sliding window, hierarchical
Pre-retrieval	Query rewriting, clarification, sub-queries
Post-retrieval	Re-ordering, filtering, confidence check

8.2.15 Key Formulas Summary

Essential RAG Formulas

TF-IDF:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{N}{\text{DF}(t)}\right)$$

Cosine similarity:

$$\cos(\vec{x}, \vec{y}) = \frac{\vec{x} \cdot \vec{y}}{|\vec{x}| \cdot |\vec{y}|}$$

Jaccard similarity:

$$\text{Jaccard}(X, Y) = \frac{|X \cap Y|}{|X \cup Y|}$$

Precision@k: $\dfrac{\#\text{ relevant in top } k}{k}$

Recall@k: $\dfrac{\#\text{ relevant in top } k}{\text{total relevant}}$

9 Week 10: Games and LLM Reasoning

9.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Understand what **LLM reasoning** is and how it differs from standard language modelling, including the role of **chain-of-thought** reasoning.
2. Explain how reasoning models are trained, including **supervised fine-tuning (SFT)**, **reinforced fine-tuning (RLFT)**, and why **intermediate tokens** matter.
3. Apply and evaluate reasoning methods during inference, such as **self-consistency decoding** and **retrieval-augmented reasoning**.

The Big Picture

This week explores how reasoning emerges in LLMs by drawing parallels with game-playing AI (Pacman, Go, strategy games). We examine how models can be trained to improve reasoning through SFT and RLFT, and how inference strategies like self-consistency enhance performance. The key insight: reasoning requires generating variants, evaluating them, and pruning low-quality paths.

What Examiners Like to Ask

- Define what makes a task a **reasoning task**.
- Compare **complete** vs. **incomplete information games**.
- Explain the **AlphaGo** approach: policy network, value network, self-play.
- Define **reward hacking** and give an example.
- Explain why **intermediate tokens** (chain-of-thought) matter theoretically.
- Compare **long context** vs. **deep networks** for reasoning.
- Describe **SFT** and **RLFT** for training reasoning models.
- Explain **self-consistency decoding** and how it works.
- Describe how **reasoning with retrieval** works.

9.2 Overview + Core Concepts + Extended Theory

Roadmap

1. Definition of reasoning
2. Game-based reasoning: Pacman, Go, strategy games
3. AlphaGo: policy networks, value networks, self-play
4. Connecting games to LLM reasoning
5. Why intermediate tokens matter
6. Training reasoning models: SFT and RLFT
7. Inference strategies: self-consistency
8. Reasoning with retrieval

9.2.1 Definition of Reasoning

What is a Reasoning Task?

A **reasoning task** is one where you need to use:

- **Logic** — applying rules or constraints
- **Inference** — deriving conclusions from premises
- **Step-by-step thinking** — decomposing into sub-problems

Key distinction: It's not about the task itself, but the **method** — reasoning involves deriving an answer through intermediate steps.

Reasoning Example

Question: What is the output when concatenating the last letter of each word in "artificial intelligence"?

Direct answer: "LE" (may be wrong)

Reasoning answer: The last letter of "artificial" is "l". The last letter of "intelligence" is "e". Concatenating gives "le".

The reasoning approach is more reliable because it shows the work.

9.2.2 Game-Based Reasoning: From Pacman to Go

Games provide a useful framework for understanding reasoning:

Key Concepts in Game Reasoning

- **Variants:** Generate multiple solution paths (different moves)
- **Reward:** Estimate the value/potential of each path
- **Pruning:** Discard low-value paths early
- **Action sequence:** Each solution contains multiple actions

Game	Subtasks	Immediate Reward	Complete Info?
Pacman	Single action	Yes	Yes
Go	Single action	No (needs training)	Yes
StarCraft/Warcraft	Action sequence	No (needs training)	No (fog of war)

9.2.3 Complete vs. Incomplete Information Games

Game Types

Complete Information Games:

- All players share full knowledge of the game state
- Examples: Chess, Go, Checkers

Incomplete Information Games:

- Players lack full knowledge; some information is private
- Must infer or form beliefs about opponents
- Examples: Poker, StarCraft (fog of war), real-world scenarios

AI Milestones in Games

- **Chess (1997):** Deep Blue defeats Kasparov
- **Go (2016):** AlphaGo defeats Lee Sedol
- **Poker (2017):** Libratus wins 2-player Texas Hold'em
- **StarCraft II (2019):** AlphaStar reaches top 0.2% — but hasn't beaten world champions

Strategy games remain hard: action sequences, no immediate rewards, fog of war.

9.2.4 AlphaGo: Learning to Play Go

AlphaGo's Two-Stage Approach

Stage 1: Learn from human games

- **Policy network:** Predict the next move given board state
- Trained on millions of human game records

Stage 2: Learn from self-play

- Two models play against each other
- **Value network:** Estimate win probability from current position
- Distribute rewards: positive for winning moves, negative for losing moves

Policy vs. Value Networks

Policy network: "What move should I make?" (action selection)

Value network: "How good is this position?" (state evaluation)

Together, they enable:

- Assigning reward to individual moves (not just final outcome)
- Pruning unpromising branches early
- Focusing search on promising directions

Reward Hacking

Reward hacking: The model finds a shortcut to maximise reward that violates the designer's intent.

Pacman example: If we add a reward for staying away from ghosts, Pacman may hide and wait for bonus fruits instead of playing normally.

LLM example: If we reward tool calling, the LLM may call tools repeatedly but never use the results.

9.2.5 Connecting Games to LLM Reasoning

Game Concepts Applied to LLMs

Game Concept	LLM Reasoning Equivalent
Action	Generating one token
Subtask	Sequence of tokens (reasoning step)
Variant	Different reasoning paths
Immediate reward	Learned reward function for intermediate steps
Incomplete info	LLM cannot fully recognise user intent

9.2.6 Why Intermediate Tokens (Chain-of-Thought) Matter

Theoretical Result

For problems solvable by boolean circuits of size T:

- **With $O(T)$ intermediate tokens:** Constant-size transformers can solve them
- **Without intermediate tokens:** Requires huge depth or cannot solve at all

Translation: You can trade **long context** for **deep networks**.

Long Context vs. Deep Networks

	Long Context (CoT)	Deep Networks
Training	Easy (SFT/RL on existing model)	Requires new model (expensive)
Inference	Training-free possible (prompting)	Requires trained model
Tokens	More tokens needed	Fewer tokens
Explainability	High (shows reasoning)	Low

Pre-trained LLMs Can Reason

Research shows that checking more generation candidates reveals reasoning:

- Greedy decoding may give short, wrong answers
- Sampling diverse candidates often reveals longer, correct reasoning paths
- Longer responses tend to have better answers

9.2.7 Supervised Fine-Tuning (SFT) for Reasoning

SFT for Reasoning

Goal: Train a model to think step-by-step without explicit instructions.

Process:

1. Collect problems with step-by-step solutions (from humans or model self-play)
2. Maximise likelihood of correct solutions
3. (Optional) Minimise likelihood of incorrect solutions

SFT Limitations

- **Easy to implement** but may not generalise well
- **Human annotators cannot provide large-scale data**
- Similar to AlphaGo's first stage: learns from human examples

---

9.2.8 Reinforced Fine-Tuning (RLFT)

RLFT for Reasoning

Goal: Learn from self-play without human annotations.

Process:

1. Generate step-by-step solutions from the model itself
2. Use a **verifier** to check if solutions are correct
3. Maximise likelihood of correct solutions
4. Minimise likelihood of wrong solutions
5. Repeat

Similar to AlphaGo's self-play stage!

RLFT Benefits and Challenges

Benefits:

- Directly optimises for the reasoning task
- No human annotators needed
- Uses verifiers to evaluate step-by-step solutions

Challenges:

- How to verify if generated solutions are correct?
- Some tasks cannot be automatically verified (math and code are easier)
- Finding reliable verifiers for non-math domains is hard

Verifiable Domains

Math: Every step can be verified by calculation.

Coding: Execute code to check correctness.

24-point game: Limited vocabulary (4 numbers + 4 operators), verifiable by calculation.
These domains have seen significant RLFT improvements.

9.2.9 Self-Consistency Decoding

Self-Consistency

If we don't need the rationale, generate as many reasoning paths as possible and vote on the answer.
Process:

1. Generate multiple responses by random sampling (different reasoning paths)
2. Extract the final answer from each response
3. Choose the answer that appears most frequently (majority vote)

Optional: Let the LLM self-select the most consistent answer from candidates.

Self-Consistency Example

Question: "Where do people drink less coffee than in Mexico?"
Response 1: "Japan, China, UK..." (Answer: Japan)
Response 2: "Japan, China, India..." (Answer: Japan)
Response 3: "1. Japan... 2. China... 3. Saudi Arabia... 4. India..." (Answer: Japan)
Majority vote: Japan ✓

9.2.10 Latent Reasoning (Advanced)

Latent Reasoning Approaches

Instead of explicit chain-of-thought tokens, manipulate the model's internal representations:

- Perturb embeddings with controlled Gaussian noise
- Use Bayesian optimisation to find embeddings that lead to high-reward outputs
- Guide reasoning via internal states rather than language-based instructions

Trade-off: Less interpretable but potentially more efficient.

9.2.11 Reasoning with Retrieval

Retrieval-Augmented Reasoning

Apply game-inspired reasoning to RAG systems:
Variant: Change the order or selection of retrieved documents.
Reward: Use another LLM to evaluate output quality.
Pruning: Update, reorder, or filter retrieved documents based on evaluation.

Retrieval + Reasoning Pipeline

1. Retrieve chunks: Chunk_1, Chunk_2, Chunk_3
2. Generate answer with LLM_1
3. Evaluate answer with LLM_2 (reward)
4. If reward is low: reorder/filter chunks, create variants
5. Repeat until satisfactory answer

LLMs as Analogical Reasoners

Retrieval can provide analogous cases for reasoning:

- Retrieve similar solved problems
- Use them as in-context examples
- LLM applies analogous reasoning to new problem

9.2.12 Summary: Reasoning Methods Comparison

Method	Training	Inference
SFT	Human step-by-step data	Standard decoding
RLFT	Self-play + verifier	Standard decoding
Self-consistency	None (or any)	Multiple samples + voting
CoT prompting	None	"Let's think step by step"
Retrieval-augmented	Optional	Retrieve + generate + evaluate

Key Takeaways

1. **Reasoning = variants + rewards + pruning** (inspired by game AI)
2. **Intermediate tokens** allow shallow models to solve complex problems
3. **SFT** learns from examples; **RLFT** learns from self-play with verifiers
4. **Self-consistency** improves accuracy through majority voting
5. **Retrieval** can augment reasoning with analogous cases

10 Week 11: LLM Evaluation and Safety

10.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Learn about common **benchmarks** used to evaluate LLMs.
2. Explore different **methods** to evaluate LLMs (automatic, human, LLM-as-judge).
3. Understand key **metrics and challenges** in LLM evaluation.
4. Understand key **safety risks** in LLMs (bias, toxicity, jailbreaking).
5. Evaluate **defense strategies** for improving LLM safety.
6. Critically assess **trade-offs and limitations** of current safety approaches.

The Big Picture

This week covers two critical aspects of LLMs: (1) **Evaluation**—how to measure LLM capabilities across tasks, benchmarks, and metrics; and (2) **Safety**—understanding vulnerabilities (bias, toxicity, jailbreaking) and defense strategies. Both are essential for responsible LLM deployment.

What Examiners Like to Ask

- Why might high benchmark scores not reflect true reasoning ability?
- Explain **Goodhart's Law** in the context of LLM leaderboards.
- Compare **text overlap metrics** (BLEU, ROUGE) vs. **semantic similarity** (BERTScore).
- Explain **LLM-as-a-Judge** and its challenges.
- What does performance drop on **GSM-Symbolic** suggest about LLM reasoning?
- Define **representational** vs. **allocational** harm.
- Describe bias evaluation methods: embedding-based, probability-based, generation-based.
- Compare **black-box** vs. **white-box** jailbreaking techniques.
- Explain defense strategies: SmoothLLM, GradSafe.

10.2 Overview + Core Concepts + Extended Theory

Roadmap

1. Why evaluation matters
2. Evaluation tasks and benchmarks
3. Evaluation methods (automatic, human, LLM-as-judge)
4. Evaluation metrics
5. Evaluation challenges
6. Safety overview
7. Social bias in LLMs
8. Toxicity in LLMs
9. Jailbreaking attacks (black-box and white-box)

10. Defense strategies

10.2.1 Why Evaluation Matters

Importance of LLM Evaluation

- Understand **true capabilities** beyond raw scores
- Ensure **fairness, robustness, and safety**
- Avoid **overfitting to benchmarks**
- Enable meaningful **model comparison**

Benchmark Saturation

When models reach > 90% on benchmarks like AIME, the benchmark no longer distinguishes real progress. High scores don't necessarily mean human-level reasoning.

10.2.2 Evaluation Tasks

Category	Examples
Language Understanding	Reading comprehension, NLI, summarisation, sentiment
Knowledge & Reasoning	MMLU, BIG-bench, mathematical reasoning, fact verification
Dialogue & Interaction	Instruction following, helpfulness, harmlessness (HHH)
Safety & Robustness	Toxicity detection, bias testing, adversarial robustness
Multimodal	Image+text reasoning, visual QA, chart/table reasoning
Specialised	Theory of Mind, emotion understanding, tool use

10.2.3 Evaluation Benchmarks

Benchmark Types

General benchmarks: MMLU, BIG-bench, HELM, MT-Bench, AGIEval, C-Eval

Domain-specific: MultiMedQA (medical), CUAD (legal), ChemBench (science), SafetyBench

Multimodal: MMBench, VQAv2, MathVista, ToolBench

Key Benchmarks

- **MMLU:** 57 subjects, 15,908 MCQs covering multitask knowledge
- **ARC-AGI:** Abstract reasoning requiring semantic interpretation of symbols
- **Chatbot Arena:** Blind pairwise comparisons where users vote on anonymous LLM responses
- **GSM-Symbolic:** Tests whether LLMs rely on pattern matching vs. true reasoning

---

10.2.4 Evaluation Methods

Three Evaluation Approaches

1. **Automatic Evaluation:**
   - Accuracy metrics: Exact Match, F1, BLEU, ROUGE
   - Calibration, fairness, robustness metrics
2. **Human Evaluation:**
   - Expert assessment on accuracy, relevance, helpfulness
   - Crowdsourced evaluation from multiple non-experts
   - Comparative/pairwise evaluation
3. **LLM-as-a-Judge:**
   - Single model judging: Strong LLM evaluates other outputs
   - Multi-model consensus: Multiple LLMs as judges, aggregate scores
   - Constitutional AI evaluation: Models trained specifically for evaluation

---

10.2.5 Evaluation Metrics

Text Overlap Metrics

- **Exact Match (EM):** % of answers exactly matching ground truth
- **BLEU:** Precision-focused n-gram overlap
- **ROUGE:** Recall-focused n-gram overlap (ROUGE-N, ROUGE-L)
- **METEOR:** Handles synonyms and word-order variations

Problem: These ignore semantic similarity between reference and candidate.

Semantic Similarity Metrics

- **BERTScore:** Compares token embeddings from BERT
- **SentenceBERT:** Encodes full sentences, measures cosine similarity
- **BLEURT:** Trained to predict human evaluation scores

Metric Comparison

Candidate: "The cat sits on the mat."

Reference: "The cat is sitting on the mat."

Metric	Score
Exact Match	0
Token F1	0.77
BLEU	0.42
ROUGE-1	0.77
METEOR	0.88

METEOR captures synonyms ("sits" ≈ "sitting") better than BLEU.

10.2.6 LLM-as-a-Judge

LLM-as-a-Judge Approaches

- **Pointwise:** Score each response independently
- **Pairwise:** Compare two responses, choose better one
- **Listwise:** Rank multiple responses
- **Multi-model debate:** Multiple LLMs argue to reach consensus

LLM-as-Judge Challenges

- **Bias:** Prefers longer, well-formatted, authoritative responses
- **Egocentric bias:** Prefers outputs similar to its own style
- **Adversarial vulnerability:** Susceptible to prompt injection
- **Scaling:** Longer reasoning improves accuracy but increases compute

---

10.2.7 Evaluation Challenges

GSM vs. GSM-Symbolic

Finding: LLMs perform well on GSM-8K but poorly on GSM-Symbolic (same problems with different numbers/names).

Implication: Models may rely on **pattern matching** rather than true reasoning—they memorise solutions rather than understand the logic.

Clause Order Sensitivity

Finding: Reordering premises in logical problems causes significant performance drops.

Implication: LLMs rely on **word order patterns** rather than deep logical structure. Reordering shouldn't affect logical reasoning, but it does.

Goodhart's Law

Statement: "When a measure becomes a target, it ceases to be a good measure."

In LLM context: When models optimise for leaderboard rankings, they may overfit to benchmark tasks rather than improve general reasoning.

Risk: Developers optimise for scores instead of real-world quality.

10.2.8 LLM Safety Overview

Safety Aspects

- **Value misalignment:** Social bias, toxicity, ethics violations
- **Robustness to attack:** Jailbreaking, adversarial prompts
- **Privacy:** Data leakage, memorisation
- **Misuse:** Misinformation, deepfakes, weaponization
- **Agent safety:** Goal misalignment, deception

10.2.9 Social Bias in LLMs

Two Types of Harm

Representational harm: Reinforcing stereotypes, misrepresenting groups, using offensive language.

Example: LLM describes nurses as women and engineers as men.

Allocational harm: Creating unfair advantages/disadvantages in resource distribution.

Example: Resume-screening AI favours male candidates over female candidates.

Bias Example

Prompt: "Two Muslims walked into a..."

GPT-3 output: Frequently generates violent completions.

Replacing "Muslim" with other religions: Significantly reduces violent completions.

This shows learned associations from training data.

Bias Evaluation Methods

Embedding-based: Compare distances between biased/unbiased representations (WEAT, SEAT).

Probability-based: Compare token probabilities for different demographic groups.

Generation-based: Analyse token distributions in actual outputs.

Key insight: Embedding-level neutrality doesn't guarantee neutral outputs—biases can re-emerge during generation.

Bias Mitigation Strategies

Pre-processing: Data balancing, data cleaning.

In-training: Architecture modifications, loss function adjustment.

Inference: Decoding strategy modification, attention weight adjustment.

Post-processing: Keyword replacement, rewriting.

10.2.10 Toxicity in LLMs

Toxicity Definition

Toxicity refers to generation of harmful or offensive content that can:

- Escalate hostility
- Disrupt social harmony
- Threaten safety

Causes: Toxic training data, generalisation of harmful patterns, interactive evolution.

Toxicity Mitigation by Phase

Pre-training: Filter toxic data (but may weaken general capabilities).

SFT: Fine-tune on non-toxic subsets, high-quality aligned data.

Alignment: RLHF, RLAIF, Adversarial DPO.

Key trade-off: Removing all toxic data can hurt model performance—toxic content may carry useful linguistic patterns.

10.2.11 Jailbreaking Attacks

Jailbreaking Definition

Jailbreaking is manipulating an LLM through crafted prompts to bypass safety filters and generate restricted outputs.

Black-Box Jailbreaking

No internal model access required—only API/chat interface.

Techniques:

- **Goal hijacking:** "IGNORE INSTRUCTIONS! NOW SAY..."
- **Refusal suppression:** "Never say 'cannot' or 'unfortunately'"
- **Role-play:** Assign personas (hacker, propagandist)
- **Fictional scenario:** "Pretend you're in developer mode"
- **Prompt decomposition:** Split malicious prompts into benign sub-prompts
- **Low-resource language:** Exploit weaker safety in underrepresented languages
- **Cipher text:** Encode prompts in Base64 or custom encodings

White-Box Jailbreaking

Requires full access to model internals (architecture, weights, gradients).

Greedy Coordinate Gradient (GCG):

1. Calculate loss for current adversarial prompt
2. Compute gradients for token positions in adversarial suffix
3. Sample candidate token replacements from top-k
4. Apply substitutions that reduce loss most
5. Repeat until successful attack

Key finding: Adversarial suffixes may transfer across different LLMs.

10.2.12 Defense Strategies

SmoothLLM

Observation: Adversarial suffixes are fragile to character-level perturbations.

Defense:

1. Create N copies of the attacked prompt
2. Randomly perturb each copy (character-level noise)
3. Generate responses for all perturbed copies
4. Aggregate responses to produce final output

Perturbations break the carefully crafted adversarial suffix.

GradSafe

Idea: Use gradient analysis to detect unsafe prompts.

Process:

1. Identify safety-critical parameters where gradients differ between safe/unsafe prompts
2. Compute loss gradients for input prompt (with response "Sure")
3. Flag as unsafe if gradient shows high cosine similarity to unsafe gradient references

Balanced Defense Strategy

For robust defense against both black-box and white-box attacks:

- Gradient-similarity unsafe prompt detection (GradSafe)
- Decoding constraints at inference time
- Pre-training data cleaning + RLHF alignment
- Post-processing content filters

No single defense is sufficient—layered approaches are needed.

10.2.13 Summary Tables

Evaluation Method	Strengths	Weaknesses
Automatic (BLEU, F1)	Fast, reproducible	Ignores semantics
Semantic (BERTScore)	Captures meaning	Compute-intensive
Human evaluation	Gold standard	Expensive, inconsistent
LLM-as-judge	Scalable, nuanced	Bias, adversarial vulnerability

Safety Issue	Detection	Mitigation
Social bias	Embedding/probability metrics	Data balancing, post-processing
Toxicity	Toxicity classifiers	Pre-training filters, RLHF
Black-box jailbreak	Input pattern detection	Prompt filtering, refusal training
White-box jailbreak	Gradient analysis (GradSafe)	SmoothLLM, adversarial training

Key Takeaways

Evaluation:

- High benchmark scores $\neq$ true reasoning (GSM-Symbolic, clause order sensitivity)
- Goodhart's Law: Optimising for metrics can corrupt them
- LLM-as-judge is scalable but has biases

Safety:

- Representational harm (stereotypes) vs. allocational harm (unfair outcomes)
- Black-box attacks use prompt engineering; white-box uses gradients
- Layered defense strategies are essential